Data Science Comprehensive Study Notes

195 Essential Concepts

Data Cleaning • EDA • Statistical Analysis • Clustering • Forecasting

Based on: retail_sales_dataset.csv | Python 3.x | scikit-learn · statsmodels · Prophet · pmdarima

Chapter 1

Data Cleaning

Import, inspect, clean, validate

37 Concept

#001 Importing Libraries

DEFINITION

The first step in any Python data science project is importing the necessary libraries. pandas provides DataFrame structures for tabular data manipulation. numpy supports numerical arrays and math operations. matplotlib and seaborn handle visualization. scipy and sklearn provide statistical and machine learning utilities.

FORMULA

N/A

WHEN TO USE

At the start of every notebook or script before any data operations.

PYTHON CODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import warnings
warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', 50)
```

Always pin library versions in production (pip freeze > requirements.txt). Import only what you need to keep namespace clean.

#002 Loading a Dataset

DEFINITION

Reading external data files into a pandas DataFrame. pd.read_csv() handles comma-separated files and supports parameters for encoding, separators, date parsing, and chunk reading. For large files, use chunksize or dtype specification to reduce memory usage.

FORMULA

N/A

WHEN TO USE

Whenever you need to bring raw data into Python for analysis.

PYTHON CODE

```
df = pd.read_csv('retail_sales_dataset.csv',
                 parse_dates=['order_date'],
                 encoding='utf-8',
                 low_memory=False)
print(df.shape)   # (rows, columns)
```

Check .shape immediately after loading. If rows/cols look wrong, inspect sep, encoding, or header parameters.

#003 Initial Dataset Inspection

DEFINITION

A quick structural audit of the DataFrame using .shape (dimensions), .dtypes (column types), .head()/.tail() (sample rows), .info() (non-null counts + memory), and .describe() (summary statistics). This reveals type mismatches, obvious missing values, and scale of the data before deeper cleaning.

FORMULA

N/A

WHEN TO USE

Immediately after loading any new dataset.

PYTHON CODE

```
print(df.shape)
print(df.dtypes)
df.head(3)
df.info()
df.describe(include='all').T
```

If a numeric column shows as object dtype, it likely contains stray characters. If max >> mean, suspect outliers.

#004 Missing Value Detection

DEFINITION

Missing values (NaN/None/NaT) are data points that were not recorded. df.isnull() returns a boolean DataFrame; .sum() along axis=0 counts missing values per column. Missing data can bias models and break statistical tests, so detection is the mandatory first cleaning step.

FORMULA

Missing count = Σ isnull(x■)

WHEN TO USE

Always — before any imputation, transformation, or modeling.

PYTHON CODE

```
missing = df.isnull().sum()
print(missing[missing > 0])
# Also check for empty strings
print((df == '').sum())
```

A column with >50% missing is usually better dropped than imputed. Check if missingness is random (MAR) or systematic (MNAR).

#005 Missing Value Percentage

DEFINITION

Expressing missing counts as a percentage of total rows gives a normalized view independent of dataset size. This helps prioritize which columns need immediate attention and informs the imputation strategy (e.g., <5% → impute; 30–50% → advanced imputation or drop; >50% → likely drop).

FORMULA

Missing % = (missing_count / n_rows) × 100

WHEN TO USE

After detecting missing values, before deciding imputation strategy.

PYTHON CODE

```
missing_pct = (df.isnull().sum() / len(df) * 100).round(2)
missing_df = pd.DataFrame({'Count': df.isnull().sum(),
                           'Percent': missing_pct})
print(missing_df[missing_df['Count'] > 0].sort_values('Percent', ascending=False))
```

Columns with 1–5% missing are safe to impute. Above 20%, imputed values introduce significant uncertainty.

#006 Missing Value Heatmap

DEFINITION

A visual representation of the pattern of missing values across the dataset. seaborn's heatmap with the boolean isnull() result shows which rows and columns are affected and whether missingness clusters in certain regions (e.g., all missing in consecutive rows → data entry issue).

FORMULA

N/A

WHEN TO USE

When you have moderate missing data and want to see structural patterns.

PYTHON CODE

```
import missingno as msno          # pip install missingno
msno.heatmap(df, figsize=(12, 6))
plt.title('Missing Value Correlation Heatmap')
plt.show()
# Or with seaborn:
sns.heatmap(df.isnull(), cbar=False, yticklabels=False, cmap='viridis')
```

If two columns consistently have missing values together, they may share a collection mechanism that failed.

#007 Duplicate Row Detection

DEFINITION

Exact duplicate rows occur when the same observation is recorded more than once. df.duplicated() returns True for all rows after the first occurrence of a duplicate. Duplicates inflate counts, distort distributions, and bias any aggregation or model trained on the data.

FORMULA

N/A

WHEN TO USE

Right after loading; especially important for transactional or survey data.

PYTHON CODE

```
n_dupes = df.duplicated().sum()
print(f'Duplicate rows: {n_dupes} ({n_dupes/len(df)*100:.1f}%)')
# Inspect duplicates
df[df.duplicated(keep=False)].sort_values(df.columns.tolist()).head(10)
```

Consider 'subset=' to check key-column duplicates (e.g., same order_id) rather than all-column duplicates.

#008 Duplicate Row Removal

DEFINITION

After detecting duplicates, df.drop_duplicates() removes all rows that are exact copies of a previous row. keep='first' (default) retains the first occurrence; keep='last' retains the last; keep=False removes all duplicates. The subset parameter allows partial-key deduplication.

FORMULA

N/A

WHEN TO USE

After confirming that duplicate rows are truly redundant (not legitimately repeated events).

PYTHON CODE

```
before = len(df)
df = df.drop_duplicates()
after = len(df)
print(f'Removed {before - after} duplicate rows')
df = df.reset_index(drop=True)
```

Always log how many rows were dropped. If the number is unexpectedly large, investigate whether the join or data pipeline created them.

#009 Categorical Value Inspection

DEFINITION

For categorical columns, .value_counts() lists each unique value and its frequency. This reveals inconsistent spellings ('Male' vs 'male' vs 'M'), extraneous whitespace, and rare categories that may need consolidation. It is the primary tool for understanding the domain and quality of categorical data.

FORMULA

N/A

WHEN TO USE

For every categorical/text column during EDA and data cleaning.

PYTHON CODE

```
for col in df.select_dtypes(include='object').columns:
    print(f'\n--- {col} ---')
    print(df[col].value_counts(dropna=False))
```

Expect fewer than ~20 unique values for a true categorical column. Hundreds of unique values may indicate a free-text or ID column.

#010 Text Standardization

DEFINITION

Textual data often has inconsistent casing, leading/trailing spaces, or mixed formatting. .str.strip() removes whitespace, .str.lower()/.upper()/.title() normalizes case. These operations are essential before any grouping, filtering, or mapping based on text values.

FORMULA

N/A

WHEN TO USE

For any text or object column before grouping, encoding, or comparison.

PYTHON CODE

```
df['gender'] = df['gender'].str.strip().str.title()
df['city'] = df['city'].str.strip().str.title()
df['order_status'] = df['order_status'].str.strip().str.lower()
# Verify
print(df['gender'].value_counts())
```

Title case ('Male', 'Female') is human-readable. Lower case is better for programmatic comparisons. Be consistent.

#011 Regex-based Cleaning

DEFINITION

Regular expressions (regex) allow pattern-based search and replacement in strings. Python's re module and pandas .str.replace(regex=True) enable removing unwanted characters (currency symbols, brackets, extra spaces), extracting substrings, or validating formats like email/phone numbers.

FORMULA

N/A

WHEN TO USE

When simple strip/lower is insufficient — e.g., mixed currency formats, phone numbers, HTML tags.

PYTHON CODE

```
import re
# Remove currency symbols and commas from a price column
df['revenue'] = df['revenue'].astype(str).str.replace(r'[$,]', '', regex=True)
df['revenue'] = pd.to_numeric(df['revenue'], errors='coerce')
# Validate email format
df['valid_email'] = df['email'].str.match(r'^[\w.-]+@[\w.-]+\.\w+$')
```

Test your regex patterns on a small sample before applying to the full column. Use raw strings (r'...') to avoid backslash issues.

#012 Date Parsing with pd.to_datetime

DEFINITION

Date columns loaded as strings must be converted to datetime dtype for time-based operations. pd.to_datetime() handles ISO format (YYYY-MM-DD) automatically. For ambiguous or mixed formats, specify format= explicitly. errors='coerce' turns unparseable values into NaT (Not a Time) for further handling.

FORMULA

N/A

WHEN TO USE

Whenever a column represents dates, timestamps, or durations.

PYTHON CODE

```
df['order_date'] = pd.to_datetime(df['order_date'],
                                  format='%Y-%m-%d',
                                  errors='coerce')
nat_count = df['order_date'].isna().sum()
print(f'Failed parses (NaT): {nat_count}')
```

Always check for NaT after conversion. A large NaT count signals mixed formats that need the dateutil approach.

#013 Mixed Format Date Handling

DEFINITION

When a date column contains multiple formats (e.g., '2023-01-15', '15/01/2023', 'January 15, 2023'), pd.to_datetime(infer_datetime_format=True) may still fail. The dateutil.parser.parse() function is a robust fallback that handles virtually any human-readable date format using a try/except approach.

FORMULA

N/A

WHEN TO USE

When a date column has 2+ date formats that cannot be handled by a single format= string.

PYTHON CODE

```
from dateutil import parser

def safe_parse(val):
    try:
        return parser.parse(str(val))
    except:
        return pd.NaT

df['order_date'] = df['order_date'].apply(safe_parse)
print(df['order_date'].isna().sum(), 'failed parses')
```

dateutil is slower than pd.to_datetime with format= but far more tolerant. Use it when you know formats are mixed.

#014 Date Validation

DEFINITION

After parsing, validate that dates fall within a plausible range for the domain. For example, order dates shouldn't be in the future or before the business was founded. Out-of-range dates are usually data entry errors and should be treated as missing (NaT) or flagged.

FORMULA

N/A

WHEN TO USE

After any date parsing step, especially in transactional or event data.

PYTHON CODE

```
min_date, max_date = pd.Timestamp('2020-01-01'), pd.Timestamp('2024-12-31')
invalid_mask = (df['order_date'] < min_date) | (df['order_date'] > max_date)
print(f'Invalid dates: {invalid_mask.sum()}')
df.loc[invalid_mask, 'order_date'] = pd.NaT
```

Domain knowledge is critical here. A 'date of birth' of 1899 for a current customer is clearly an error.

#015 Date Feature Extraction

DEFINITION

Once a column is datetime dtype, the .dt accessor unlocks dozens of time-based attributes: .dt.year, .dt.month, .dt.day, .dt.dayofweek (0=Monday), .dt.quarter, .dt.is_month_end, .dt.week. These derived features are essential for time-series analysis, seasonality detection, and date-based grouping.

FORMULA

N/A

WHEN TO USE

After parsing dates, to create time-based features for EDA or modeling.

PYTHON CODE

```
df['year']        = df['order_date'].dt.year
df['month']       = df['order_date'].dt.month
df['day_of_week'] = df['order_date'].dt.dayofweek
df['quarter']     = df['order_date'].dt.quarter
df['is_weekend']  = df['day_of_week'].isin([5, 6]).astype(int)
```

day_of_week=0 is Monday. Weekend flag (5,6 = Sat/Sun) is a common feature in retail sales models.

#016 Numeric Type Coercion

DEFINITION

Columns that should be numeric but were loaded as strings (due to commas, currency symbols, or mixed entries) must be coerced. pd.to_numeric(errors='coerce') converts to float/int and turns non-convertible values into NaN, which can then be handled by imputation.

FORMULA

N/A

WHEN TO USE

When numeric columns have dtype=object after loading.

PYTHON CODE

```
num_cols = ['age', 'quantity', 'unit_price', 'revenue']
for col in num_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')
    print(f'{col}: {df[col].isna().sum()} coerced to NaN')
```

After coercion, run df.describe() on those columns to verify the range looks sensible.

#017 Categorical Dtype Conversion

DEFINITION

Converting object columns with limited unique values to pandas Categorical dtype (.astype('category')) reduces memory usage significantly and enables ordered comparisons. Category columns also render more efficiently in groupby and value_counts operations.

FORMULA

N/A

WHEN TO USE

When a column has fewer than ~50 unique values relative to dataset size.

PYTHON CODE

```
cat_cols = ['gender', 'category', 'city', 'order_status', 'payment_method']
for col in cat_cols:
    df[col] = df[col].astype('category')
print(df.memory_usage(deep=True).sum() / 1024**2, 'MB')
```

Converting to category can cut memory by 50–80% for high-cardinality repetitive string columns.

#018 Mean Imputation

DEFINITION

Replacing missing numeric values with the column mean. Simple and fast, but it reduces variance and distorts the distribution shape. Only appropriate when data is Missing Completely At Random (MCAR) and the column is roughly symmetric (not skewed).

FORMULA

x■ = (1/n) Σx■ (mean of non-missing values)

WHEN TO USE

For approximately normal, low-missingness (<5%) numeric columns.

PYTHON CODE

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy='mean')
df['age_imputed'] = imp.fit_transform(df[['age']])
# Or simply:
df['age'].fillna(df['age'].mean(), inplace=True)
```

Never use mean imputation on skewed distributions — it pulls values toward the center and underestimates the spread.

#019 Median Imputation

DEFINITION

Replacing missing numeric values with the column median. The median is the 50th percentile (middle value when sorted) and is robust to outliers. For skewed distributions or data with outliers, median imputation is almost always preferred over mean imputation.

FORMULA

x■ = median(x■, x■, ..., x■)

WHEN TO USE

For skewed numeric columns or any column with outliers.

PYTHON CODE

```
skewed_cols = ['revenue', 'quantity', 'days_to_ship']
for col in skewed_cols:
    median_val = df[col].median()
    df[col].fillna(median_val, inplace=True)
    print(f'{col} median fill: {median_val:.2f}')
```

■ ***Check skewness (df.skew()) before choosing mean vs median. |skew| > 1 → use median.***

#020 Mode Imputation

DEFINITION

Replacing missing categorical (or discrete numeric) values with the most frequent value (mode). The mode is the value that appears most often in the distribution. For nominal categorical data, it is the only sensible central tendency measure.

FORMULA

x■ = argmax(count(x■))

WHEN TO USE

For categorical, binary, or low-cardinality integer columns.

PYTHON CODE

```
cat_cols = ['gender', 'category', 'payment_method']
for col in cat_cols:
    mode_val = df[col].mode()[0]
    df[col].fillna(mode_val, inplace=True)
    print(f'{col} mode: {mode_val}')
```

■ ***Be careful with mode imputation if the mode accounts for <40% of values — it may introduce bias toward the dominant category.***

#021 Constant / Flag Imputation

DEFINITION

Filling missing values with a sentinel constant such as 'Unknown', -1, or 0. For categorical columns, 'Unknown' explicitly captures the fact that the value was missing, preserving that information for downstream analysis or modeling. For numeric columns, -1 or -999 can serve as a flag when missingness itself is informative.

FORMULA

N/A

WHEN TO USE

When missingness itself is a signal worth preserving, or when the column has too many uniques for mode imputation to make sense.

PYTHON CODE

```
df['promo_code'].fillna('No Promo', inplace=True)
df['return_flag'].fillna(0, inplace=True)
# Verify
print(df['promo_code'].value_counts())
```

Creating an explicit 'Unknown' category allows a model to learn that missingness itself predicts the target.

#022 Forward Fill and Backward Fill

DEFINITION

Forward fill (ffill) propagates the last observed value forward to fill subsequent NaNs; backward fill (bfill) does the reverse. These are most appropriate for time-ordered data where adjacent values are correlated (e.g., sensor readings, stock prices, panel data).

FORMULA

N/A

WHEN TO USE

For time-series or ordered data where temporal continuity is expected.

PYTHON CODE

```
# Sort by time first
df_ts = df.sort_values('order_date')
df_ts['price_trend'].fillna(method='ffill', inplace=True)
# Alternatively:
df_ts['price_trend'] = df_ts['price_trend'].ffill().bfill()
```

Never use ffill/bfill on unordered data — it will propagate incorrect values. Ensure the DataFrame is sorted by the time column first.

#023 Group-wise Median Imputation

DEFINITION

Instead of using the overall column median, impute each missing value with the median of its group (e.g., impute age with the median age of the same gender and category). This is more accurate than global imputation when sub-groups have different distributions and is done with .groupby().transform('median').

FORMULA

x■■ = median(x | group(i))

WHEN TO USE

When the column's distribution varies meaningfully across a categorical grouping variable.

PYTHON CODE

```
df['age'] = df.groupby(['gender', 'category'])['age']                .transform(lambda x: x.fillna(x.median()))
# Handle any remaining NaN (groups with all-NaN)
df['age'].fillna(df['age'].median(), inplace=True)
```

Group-wise imputation can reduce imputation error by 20–40% compared to global imputation when groups differ significantly.

#024 KNN and Iterative Imputation

DEFINITION

KNN Imputation replaces each missing value with the weighted average of its k nearest neighbors based on non-missing features. Iterative Imputation (MICE — Multiple Imputation by Chained Equations) models each column with missing values as a function of the other columns, iterating until convergence. Both are more accurate than univariate methods for data not missing completely at random.

FORMULA

KNN: x■ = Σ(w■x■) / Σw■ where w■ = 1/distance

WHEN TO USE

When missingness is moderate (5–30%) and columns are correlated.

PYTHON CODE

```
from sklearn.impute import KNNImputer
num_df = df.select_dtypes(include=np.number)
imputer = KNNImputer(n_neighbors=5)
df_imputed = pd.DataFrame(imputer.fit_transform(num_df),
                          columns=num_df.columns)
```

KNN imputation preserves multivariate relationships better than univariate methods but is computationally expensive for large datasets.

#025 IQR Outlier Detection

DEFINITION

The Interquartile Range (IQR) method defines outliers as values that fall below Q1 – 1.5×IQR or above Q3 + 1.5×IQR. Q1 is the 25th percentile, Q3 the 75th percentile, and IQR = Q3 – Q1. This is the standard method used in box plots and is robust to the distribution's shape.

FORMULA

Lower fence = Q1 – 1.5×IQR
Upper fence = Q3 + 1.5×IQR
IQR = Q3 – Q1

WHEN TO USE

For any numeric column — especially effective for skewed distributions.

PYTHON CODE

```
def iqr_outliers(series):
    Q1, Q3 = series.quantile(0.25), series.quantile(0.75)
    IQR = Q3 - Q1
    mask = (series < Q1 - 1.5*IQR) | (series > Q3 + 1.5*IQR)
    return mask

for col in ['age', 'quantity', 'revenue']:
    n = iqr_outliers(df[col]).sum()
    print(f'{col}: {n} outliers ({n/len(df)*100:.1f}%)')
```

~3–5% outliers is typical. More than 10% suggests the 'outliers' may actually be a legitimate sub-population.

#026 Z-Score Outlier Detection

DEFINITION

The Z-score measures how many standard deviations a data point is from the mean. Values with |Z| > 3 are conventionally considered outliers (covering 99.7% of a normal distribution). Z-score method assumes normality; it performs poorly on skewed distributions.

FORMULA

Z = (x – μ) / σ
Outlier if |Z| > 3

WHEN TO USE

For approximately normally distributed columns without heavy tails.

PYTHON CODE

```
from scipy import stats
z_scores = np.abs(stats.zscore(df['age'].dropna()))
outlier_mask = z_scores > 3
print(f'Z-score outliers: {outlier_mask.sum()}')
# On DataFrame:
z_df = df[['age','revenue']].apply(stats.zscore)
df_clean = df[(np.abs(z_df) < 3).all(axis=1)]
```

Z-score is sensitive to outliers in the mean and std calculation itself. For robustness on non-normal data, prefer IQR or Modified Z-Score.

#027 Modified Z-Score (MAD Method)

DEFINITION

The Modified Z-Score replaces mean and standard deviation with the more robust median and Median Absolute Deviation (MAD). This makes it resistant to the influence of the very outliers being detected. A common threshold is |Modified Z| > 3.5.

FORMULA

MAD = median(|x■ – x■|)
Modified Z = 0.6745 × (x■ – x■) / MAD

WHEN TO USE

For heavy-tailed or skewed distributions where Z-score performs poorly.

PYTHON CODE

```
def modified_zscore(series):
    median = series.median()
    mad = (series - median).abs().median()
    return 0.6745 * (series - median) / (mad + 1e-9)

mz = modified_zscore(df['revenue'])
outliers = df[mz.abs() > 3.5]
print(f'Modified Z outliers: {len(outliers)}')
```

■ ***The factor 0.6745 scales MAD to be consistent with the standard deviation for normal data.***

#028 Outlier Capping / Clipping

DEFINITION

Instead of removing outliers, capping (also called Winsorization or clipping) replaces extreme values with the fence values (Q1–1.5×IQR and Q3+1.5×IQR). This retains all rows while reducing the distortive influence of extreme values on statistics and models. np.clip() or pd.Series.clip() perform this operation.

FORMULA

x_capped = max(lower_fence, min(x, upper_fence))

WHEN TO USE

When you want to retain all rows but reduce the influence of extreme values.

PYTHON CODE

```
for col in ['age', 'revenue', 'quantity']:
    Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
    df[col] = df[col].clip(lower=lower, upper=upper)
    print(f'{col}: clipped to [{lower:.1f}, {upper:.1f}]')
```

■ ***Capping is preferred over removal in regression tasks since it preserves sample size and avoids information loss.***

#029 Winsorizing with SciPy

DEFINITION

scipy.stats.mstats.winsorize() clips a specified percentile from both tails of a distribution in a single function call. Unlike np.clip() which requires manual fence calculation, winsorize takes the trim fraction directly (e.g., limits=(0.01, 0.01) removes the bottom and top 1%).

FORMULA

Winsorize at p%: replace values below the p-th percentile with that percentile value

WHEN TO USE

When you want to trim a fixed percentage from both tails without manually computing fences.

PYTHON CODE

```
from scipy.stats.mstats import winsorize
df['revenue_w'] = winsorize(df['revenue'].fillna(df['revenue'].median()),
                            limits=[0.02, 0.02])
# 2% trimmed from each tail
print(df['revenue_w'].describe())
```

limits=(0.05, 0.05) is a common choice. Larger limits reduce tail influence more but discard more true variation.

#030 Outlier Removal

DEFINITION

Completely removing rows where one or more columns contain outlier values. This is appropriate when outliers are confirmed data errors (impossible values like age=300), but should be used cautiously on genuine extreme values (e.g., a very high-value customer). Always document how many rows were removed.

FORMULA

N/A

WHEN TO USE

Only when outliers are definitively erroneous, not just extreme.

PYTHON CODE

```
# Remove rows where age > 100 or age < 0
df = df[(df['age'] >= 0) & (df['age'] <= 100)]
# Remove rows with quantity < 0
df = df[df['quantity'] >= 0]
print(f'Remaining rows: {len(df)}')
```

If removing outliers changes the analysis conclusion, that conclusion was fragile to begin with. Report both with and without outliers.

#03 1 Column Renaming

DEFINITION

Renaming columns to follow consistent naming conventions (snake_case, no spaces, lowercase) makes code more readable and avoids errors when accessing columns with df.column_name syntax. .rename(columns={...}) takes a dictionary mapping old names to new names.

FORMULA

N/A

WHEN TO USE

When columns have spaces, mixed case, or ambiguous names.

PYTHON CODE

```
df = df.rename(columns={
    'Order ID': 'order_id',
    'Customer Name': 'customer_name',
    'Ship Date': 'ship_date',
    'Unit Price': 'unit_price'
})
# Or snake_case all columns:
df.columns = df.columns.str.lower().str.replace(' ', '_')
```

Establish a naming convention at project start. Snake_case (lowercase with underscores) is the Python standard.

#03 2 Derived Column Creation

DEFINITION

Creating new columns from existing ones to capture domain-relevant information. Examples include: days_to_ship = ship_date – order_date, profit_margin = (revenue – cost) / revenue, age_group = pd.cut(age, bins). Derived features often carry more predictive power than raw columns.

FORMULA

N/A

WHEN TO USE

Whenever domain knowledge suggests a meaningful relationship between existing columns.

PYTHON CODE

```
df['days_to_ship'] = (df['ship_date'] - df['order_date']).dt.days
df['profit_margin'] = (df['revenue'] - df['cost']) / df['revenue']
df['age_group'] = pd.cut(df['age'],
                         bins=[0,18,35,50,65,100],
                         labels=['<18','18-35','36-50','51-65','65+'])
```

Always validate derived columns: check for negative values (days_to_ship < 0 means ship before order — impossible).

#033 Data Quality Report

DEFINITION

A consolidated summary of all data quality issues found: missing values, duplicates, outliers, type errors, and invalid ranges. This report serves as documentation for the cleaning decisions made and provides a baseline for tracking data quality improvements over time.

FORMULA

N/A

WHEN TO USE

At the end of the exploration phase, before cleaning — and again after cleaning for comparison.

PYTHON CODE

```
def data_quality_report(df):
    report = pd.DataFrame({
        'dtype': df.dtypes,
        'missing': df.isnull().sum(),
        'missing_pct': (df.isnull().sum()/len(df)*100).round(2),
        'unique': df.nunique(),
        'min': df.min(numeric_only=True),
        'max': df.max(numeric_only=True)
    })
    return report
print(data_quality_report(df))
```

A good data quality report is the foundation of reproducible analysis — it shows stakeholders exactly what was cleaned and why.

#034 Before/After Comparison

DEFINITION

Comparing summary statistics and distributions before and after cleaning operations validates that the cleaning had the intended effect. Key checks: shape (rows/cols), missing counts, outlier counts, and statistical summary (mean, median, std). A side-by-side comparison table is the most communicative format.

FORMULA

N/A

WHEN TO USE

After each major cleaning step to verify correctness.

PYTHON CODE

```
print('BEFORE cleaning:')
print(f'Rows: {before_shape[0]}, Missing: {before_missing}')
print(f'Revenue mean: {before_rev_mean:.2f}')

print('\nAFTER cleaning:')
print(f'Rows: {df.shape[0]}, Missing: {df.isnull().sum().sum()}')
print(f'Revenue mean: {df.revenue.mean():.2f}')
```

If the mean changed dramatically after imputation, the imputed values may be pulling it too far. Investigate the imputation logic.

#035 Saving Clean Dataset

DEFINITION

After all cleaning steps, save the clean DataFrame to a new CSV file. This separates raw and processed data, allows re-running analysis from the clean checkpoint, and ensures reproducibility. Always save to a different filename from the raw data to preserve the original.

FORMULA

N/A

WHEN TO USE

After completing all cleaning steps, before beginning EDA or modeling.

PYTHON CODE

```
df.to_csv('retail_sales_clean.csv', index=False, encoding='utf-8')
print(f'Saved clean dataset: {df.shape}')
# Verify:
df_check = pd.read_csv('retail_sales_clean.csv')
print(df_check.shape, df_check.isnull().sum().sum(), 'missing')
```

Use index=False to avoid saving the DataFrame index as an extra column. Add a timestamp to the filename for versioned datasets.

#036 Data Type Validation

DEFINITION

Programmatically checking that each column has the expected dtype after all cleaning operations. This catches silent errors (e.g., a numeric column that silently remained as object due to a missed coercion step). A validation dictionary maps column names to expected types.

FORMULA

N/A

WHEN TO USE

At the end of the cleaning pipeline as a quality gate.

PYTHON CODE

```
expected_types = {
    'order_date': 'datetime64[ns]',
    'age': 'float64',
    'revenue': 'float64',
    'quantity': 'float64',
    'gender': 'category'
}
for col, expected in expected_types.items():
    actual = str(df[col].dtype)
    status = '✓' if actual == expected else '✗'
    print(f'{status} {col}: {actual} (expected: {expected})')
```

Automate this validation in production pipelines to catch upstream data changes early.

#037 Final Validation Checks

DEFINITION

A comprehensive post-cleaning validation that confirms: no remaining missing values, no duplicate rows, all dtypes as expected, all values within valid ranges, and row/column counts match expectations. This is the final quality gate before the clean dataset is used for analysis or modeling.

FORMULA

N/A

WHEN TO USE

As the last step in any data cleaning pipeline.

PYTHON CODE

```
assert df.isnull().sum().sum() == 0, 'Still has missing values!'
assert df.duplicated().sum() == 0, 'Still has duplicates!'
assert (df['age'] >= 0).all() & (df['age'] <= 120).all()
assert (df['revenue'] >= 0).all()
assert (df['quantity'] > 0).all()
print('All validation checks passed!')
print(df.info())
```

Use Python assert statements to turn validation checks into automated tests. In production, raise meaningful exceptions instead of asserts.

#038 Descriptive Statistics (.describe())

DEFINITION

pandas .describe() computes count, mean, std, min, 25th/50th/75th percentiles, and max for all numeric columns in one call. For categorical columns, use describe(include='object') to get count, unique, top, and freq. This is the fastest way to understand the central tendency and spread of every variable.

FORMULA

mean = Σx■/n | std = √(Σ(x■–x■)²/(n–1))

WHEN TO USE

First thing after loading the clean dataset — before any visualization.

PYTHON CODE

```
# Numeric summary
print(df.describe().T.round(2))
# Categorical summary
print(df.describe(include='object').T)
# All columns
print(df.describe(include='all').T)
```

Watch for: mean >> median (positive skew), std > mean (high variability), min or max that seems impossible.

#039 Custom Aggregation (.agg())

DEFINITION

.groupby().agg() allows computing multiple aggregation functions simultaneously — e.g., mean, median, count, and std in a single call. You can pass a dictionary mapping column names to lists of functions, or use named aggregations with .agg(alias=('col','func')) syntax for clean output column names.

FORMULA

N/A

WHEN TO USE

When you need grouped summaries with multiple statistics (e.g., revenue by region: mean, total, count).

PYTHON CODE

```
summary = df.groupby('category').agg(
    avg_revenue=('revenue', 'mean'),
    total_revenue=('revenue', 'sum'),
    order_count=('order_id', 'count'),
    avg_age=('age', 'median')
).round(2)
print(summary.sort_values('total_revenue', ascending=False))
```

Named aggregations (pandas ≥0.25) produce clean column names. Always check if groupby result makes business sense.

#040 Skewness and Kurtosis

DEFINITION

Skewness measures the asymmetry of a distribution. Positive skew (right tail) means the mean > median; negative skew (left tail) means mean < median. Kurtosis measures tail heaviness relative to a normal distribution (excess kurtosis=0). High kurtosis (leptokurtic) means heavy tails with extreme values more common than normal.

FORMULA

Skewness = (1/n) Σ((x■–x■)/σ)³
Kurtosis = (1/n) Σ((x■–x■)/σ)■ – 3 (excess)

WHEN TO USE

To quantify distribution shape and decide on transformations or robust statistics.

PYTHON CODE

```
skew_kurt = df.select_dtypes(include=np.number).agg(['skew','kurt'])
print(skew_kurt.T)
# Rule of thumb: |skew| > 1 → significantly skewed
# |kurtosis| > 3 → heavy tails
```

Skew > 1: consider log transform. Kurtosis > 3: outliers are likely influential.

#041 Frequency Distribution Table

DEFINITION

A frequency table shows how many times each value (or value range) appears in the data. For categorical variables, it is a direct count table. For continuous variables, pd.cut() or pd.qcut() creates bins and value_counts() gives the distribution. Essential for understanding the raw composition of your data.

FORMULA

Frequency = count of observations in category/bin
Relative frequency = frequency / n

WHEN TO USE

For categorical columns and when you need binned summaries of continuous columns.

PYTHON CODE

```
# Categorical frequency
print(df['category'].value_counts(normalize=True).round(3) * 100)
# Binned continuous
df['age_bin'] = pd.cut(df['age'], bins=[0,18,35,50,65,100],
                       labels=['<18','18-35','36-50','51-65','65+'])
print(df['age_bin'].value_counts().sort_index())
```

Imbalanced frequency tables (one category >> others) often indicate the need for stratified sampling in modeling.

#042 Histogram

DEFINITION

A histogram displays the frequency distribution of a continuous variable by dividing the range into equal-width bins and counting observations per bin. The bin width (or number of bins) significantly affects interpretation — too few bins hide structure; too many bins create noise. Sturges' Rule (k = 1 + log■n) or sqrt(n) are common heuristics.

FORMULA

Bin width = (max – min) / k

Stirges: k = 1 + log■(n)

WHEN TO USE

For any continuous numeric variable to understand its distribution shape.

PYTHON CODE

```
fig, axes = plt.subplots(2, 3, figsize=(14, 8))
num_cols = ['age', 'revenue', 'quantity', 'unit_price', 'days_to_ship', 'profit_margin']
for ax, col in zip(axes.flatten(), num_cols):
    df[col].hist(bins=30, ax=ax, color='steelblue', edgecolor='white')
    ax.set_title(col)
plt.tight_layout(); plt.show()
```

■ ***Bell shape → normal. Right tail → positive skew (log transform may help). Bimodal → two sub-populations exist.***

#043 KDE Plot (Kernel Density Estimate)

DEFINITION

A KDE plot is a smoothed continuous estimate of the probability density function, created by placing a kernel (usually Gaussian) at each data point and summing them. It avoids the bin-width sensitivity of histograms and enables direct visual comparison of distributions between groups on the same axis.

FORMULA

f■(x) = (1/nh) Σ K((x–x■)/h)

h = bandwidth (controls smoothness)

WHEN TO USE

When comparing distributions between groups or when you want a smooth representation of the distribution.

PYTHON CODE

```
fig, ax = plt.subplots(figsize=(10, 5))
for gender in df['gender'].unique():
    subset = df[df['gender'] == gender]['revenue'].dropna()
    subset.plot.kde(ax=ax, label=gender, linewidth=2)
ax.set_xlabel('Revenue'); ax.legend()
ax.set_title('Revenue Distribution by Gender (KDE)')
plt.show()
```

■ ***Overlapping KDE curves indicate similar distributions. Non-overlapping peaks suggest the groups differ significantly.***

#044 Box Plot

DEFINITION

A box plot (box-and-whisker plot) visualizes the five-number summary: minimum, Q1, median, Q3, and maximum — plus individual outlier points beyond 1.5×IQR. It compactly conveys spread, skewness, and outlier presence. Multiple box plots side-by-side are ideal for comparing a numeric variable across categories.

FORMULA

Box: [Q1, Q3] | Whiskers: [Q1–1.5×IQR, Q3+1.5×IQR]
Outliers: points outside whiskers

WHEN TO USE

Comparing distributions of a numeric variable across categories.

PYTHON CODE

```
plt.figure(figsize=(12, 5))
order = df.groupby('category')['revenue'].median().sort_values().index
sns.boxplot(data=df, x='category', y='revenue', order=order,
            palette='Set2')
plt.title('Revenue Distribution by Category')
plt.xticks(rotation=45); plt.tight_layout(); plt.show()
```

A notch (notched boxplot) shows a 95% CI around the median. Non-overlapping notches indicate significant difference between medians.

#045 Violin Plot

DEFINITION

A violin plot combines a box plot with a KDE curve on each side, providing more information about the distribution shape (bimodality, skewness) than a box plot alone. Wider sections indicate higher data density. Inner box plots or quartile lines can be overlaid for reference.

FORMULA

N/A (combination of KDE + box summary)

WHEN TO USE

When you want richer distribution information than a box plot provides, especially to detect bimodality.

PYTHON CODE

```
plt.figure(figsize=(12, 5))
sns.violinplot(data=df, x='category', y='revenue',
               palette='husl', inner='quartile', cut=0)
plt.title('Revenue Distribution by Category (Violin)')
plt.xticks(rotation=45); plt.tight_layout(); plt.show()
```

A violin with two bulges (bimodal) suggests two distinct sub-populations within that category.

#046 Bar Chart / Count Plot

DEFINITION

A bar chart displays the frequency or aggregate value of a categorical variable. seaborn countplot() shows raw counts; barplot() shows the mean (with CI) of a numeric variable per category. Horizontal orientation improves readability for many categories. Always sort by value for quick insights.

FORMULA

N/A

WHEN TO USE

For categorical frequency distributions or category-level mean comparisons.

PYTHON CODE

```
plt.figure(figsize=(10, 5))
order = df['category'].value_counts().index
sns.countplot(data=df, x='category', order=order, palette='Blues_d')
plt.title('Order Count by Category')
plt.xticks(rotation=30); plt.tight_layout(); plt.show()
```

If one category dominates (>50%), consider whether the dataset is representative of the true population.

#047 Pie Chart

DEFINITION

A pie chart shows the proportional composition of a categorical variable as slices of a circle. While visually intuitive for small numbers of categories (≤5), pie charts are generally less precise than bar charts for comparing proportions. For many slices, an exploded pie or a bar chart is more readable.

FORMULA

Slice angle = (category_count / total) × 360°

WHEN TO USE

For showing proportional composition when there are ≤5 categories.

PYTHON CODE

```
revenue_by_cat = df.groupby('category')['revenue'].sum()
plt.figure(figsize=(8, 8))
plt.pie(revenue_by_cat, labels=revenue_by_cat.index,
        autopct='%1.1f%%', startangle=90,
        colors=sns.color_palette('pastel'))
plt.title('Revenue Share by Category')
plt.show()
```

Avoid pie charts with >6 slices. Explode the largest or most important slice for emphasis.

#048 Scatter Plot

DEFINITION

A scatter plot visualizes the relationship between two continuous variables by placing a dot for each observation at coordinates (x, y). Patterns reveal correlation direction (positive/negative/none), linearity, clustering, and outliers. Adding a third variable via color (hue) or size (size) creates a bubble/colored scatter plot.

FORMULA

N/A

WHEN TO USE

To explore the relationship between two numeric variables.

PYTHON CODE

```
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='age', y='revenue',
                hue='gender', alpha=0.6, s=40,
                palette={'Male':'steelblue','Female':'coral'})
plt.title('Age vs Revenue by Gender')
# Add regression line:
sns.regplot(data=df, x='age', y='revenue', scatter=False,
            color='black', line_kws={'linewidth':1.5})
plt.show()
```

A funnel-shaped scatter (variance increases with x) signals heteroscedasticity — a violation of OLS regression assumptions.

#049 Line Plot

DEFINITION

A line plot connects data points with lines and is most appropriate for showing trends over an ordered variable — typically time. For time series, resample the data to a desired frequency (daily, monthly) and plot the aggregated values. Multiple lines on the same axes enable comparison of trends across groups.

FORMULA

N/A

WHEN TO USE

For time-ordered data, trends over dates, or any data with a meaningful x-axis ordering.

PYTHON CODE

```
monthly = df.groupby(df['order_date'].dt.to_period('M'))['revenue']              .sum().reset_index()
monthly['order_date'] = monthly['order_date'].dt.to_timestamp()
plt.figure(figsize=(14, 5))
plt.plot(monthly['order_date'], monthly['revenue'], marker='o', ms=4)
plt.title('Monthly Revenue Trend'); plt.grid(alpha=0.3)
plt.show()
```

A steadily rising line = upward trend. Repeating peaks and troughs = seasonality. Sudden drops = anomalies to investigate.

#050 Pair Plot (seaborn.pairplot)

DEFINITION

A pair plot creates a grid of scatter plots for every pair of numeric variables, with univariate distributions (histograms or KDE) on the diagonal. It is an efficient way to see all pairwise relationships, detect correlations, and identify clusters in a single visualization. The hue parameter colors points by a categorical variable.

FORMULA

N/A

WHEN TO USE

For datasets with 3–8 numeric variables to quickly survey all pairwise relationships.

PYTHON CODE

```
num_cols = ['age', 'revenue', 'quantity', 'unit_price', 'days_to_ship']
pair_df = df[num_cols + ['gender']].dropna().sample(500, random_state=42)
g = sns.pairplot(pair_df, hue='gender', diag_kind='kde',
                 plot_kws={'alpha':0.5, 's':20},
                 palette={'Male':'steelblue','Female':'coral'})
g.fig.suptitle('Pairplot of Numeric Variables', y=1.01)
plt.show()
```

Off-diagonal patterns reveal correlations. Diagonal KDE shapes reveal distributions. Color-separated clusters indicate group differences.

#051 Correlation Heatmap

DEFINITION

A heatmap of the correlation matrix displays the Pearson (or Spearman) correlation coefficient between every pair of numeric variables using a color gradient. Values range from –1 (perfect negative correlation) through 0 (no correlation) to +1 (perfect positive correlation). Masking the upper triangle removes redundancy.

FORMULA

r = Σ((x■–x■)(y■–■)) / (n–1)·s■·s■

WHEN TO USE

To identify strongly correlated features before modeling (multicollinearity detection) and to generate hypotheses.

PYTHON CODE

```
corr = df.select_dtypes(include=np.number).corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
plt.figure(figsize=(12, 9))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f',
            cmap='RdBu_r', center=0, vmin=-1, vmax=1,
            linewidths=0.5, square=True)
plt.title('Pearson Correlation Matrix')
plt.show()
```

|r| > 0.7 → strong correlation. Two predictors with |r| > 0.8 between them signal multicollinearity — consider removing one.

#052 Pivot Table Heatmap

DEFINITION

A pivot table aggregates data into a 2D matrix indexed by two categorical variables. Visualizing this matrix as a heatmap reveals interaction effects — e.g., how revenue varies by both region AND product category. pd.pivot_table() builds the table; sns.heatmap() renders it.

FORMULA

cell(r,c) = agg_func(values where row_var=r AND col_var=c)

WHEN TO USE

To explore the interaction between two categorical variables on a numeric outcome.

PYTHON CODE

```
pivot = pd.pivot_table(df, values='revenue', aggfunc='mean',
                       index='category', columns='gender')
plt.figure(figsize=(8, 5))
sns.heatmap(pivot, annot=True, fmt='.0f', cmap='YlOrRd',
            linewidths=0.5)
plt.title('Avg Revenue: Category × Gender')
plt.show()
```

High-value cells at specific intersections reveal target segments (e.g., Female + Electronics = highest revenue cell).

#053 Grouped Bar Chart

DEFINITION

A grouped (clustered) bar chart displays bars for multiple sub-groups side-by-side within each category group. It allows comparing both within-group variation and between-group patterns simultaneously. seaborn barplot() with a hue parameter creates grouped bars.

FORMULA

N/A

WHEN TO USE

When comparing a numeric metric across two categorical dimensions simultaneously.

PYTHON CODE

```
plt.figure(figsize=(12, 5))
sns.barplot(data=df, x='category', y='revenue',
            hue='gender', estimator='mean',
            palette={'Male':'steelblue','Female':'coral'},
            capsize=0.05)
plt.title('Mean Revenue by Category and Gender')
plt.xticks(rotation=30); plt.legend(title='Gender')
plt.tight_layout(); plt.show()
```

Error bars (CI) that don't overlap indicate a statistically significant difference between groups.

#054 Stacked Bar Chart

DEFINITION

A stacked bar chart shows the total value of a category as a bar, with each bar subdivided into segments representing sub-category contributions. This reveals both the total magnitude and the compositional breakdown simultaneously. Good for part-to-whole relationships across categories.

FORMULA

N/A

WHEN TO USE

When showing both total values and proportional composition across categories.

PYTHON CODE

```
pivot = df.groupby(['category','gender'])['revenue'].sum().unstack()
pivot.plot(kind='bar', stacked=True, figsize=(10, 6),
           color=['steelblue','coral'])
plt.title('Total Revenue by Category (Stacked by Gender)')
plt.xticks(rotation=30); plt.ylabel('Revenue')
plt.tight_layout(); plt.show()
```

Compare bar heights for total magnitude and segment proportions for composition. Watch for categories where one gender dominates.

#055 Area Chart

DEFINITION

An area chart is a line chart with the area below the line filled in. For multiple groups, a stacked area chart shows cumulative values over time. It communicates trends and part-to-whole evolution together. Best for time-series data with 2–4 non-overlapping series.

FORMULA

N/A

WHEN TO USE

For visualizing cumulative time-series composition (e.g., revenue by category over months).

PYTHON CODE

```
monthly_cat = df.groupby([df['order_date'].dt.to_period('M'), 'category'])['revenue']
monthly_cat.index = monthly_cat.index.to_timestamp()
monthly_cat.plot(kind='area', stacked=True, figsize=(14, 6),
                 alpha=0.75, colormap='tab10')
plt.title('Monthly Revenue by Category (Stacked Area)')
plt.tight_layout(); plt.show()
```

A growing stacked area with stable proportions = overall growth with consistent mix. Changing proportions = shifting product mix.

#056 Bubble Chart

DEFINITION

A bubble chart extends the scatter plot by encoding a third numeric variable as the size of each bubble. A fourth variable can be encoded as color. This enables 4-dimensional visualization in 2D space. Common use: x=age, y=revenue, bubble_size=quantity, color=category.

FORMULA

bubble_area ∝ third_variable

WHEN TO USE

When you want to compare three or four dimensions simultaneously in a single plot.

PYTHON CODE

```
sample = df.dropna().sample(300, random_state=42)
plt.figure(figsize=(12, 7))
scatter = plt.scatter(sample['age'], sample['revenue'],
                      s=sample['quantity']*15,
                      c=pd.Categorical(sample['category']).codes,
                      alpha=0.6, cmap='tab10')
plt.xlabel('Age'); plt.ylabel('Revenue')
plt.title('Age vs Revenue (size=Quantity, color=Category)')
plt.colorbar(scatter, label='Category')
plt.show()
```

Clusters of large bubbles indicate high-quantity, high-revenue customer segments — prime targets for loyalty programs.

#057 Time Series Resampling (.resample())

DEFINITION

Resampling changes the frequency of time-series data. Downsampling (e.g., daily → monthly) aggregates finer data into coarser intervals. Upsampling (e.g., monthly → daily) requires interpolation. pd.resample() with a DatetimeIndex is the standard tool. Common offset strings: 'D' (day), 'W' (week), 'M' (month), 'Q' (quarter), 'A' (year).

FORMULA

N/A

WHEN TO USE

To change time-series granularity for trend analysis or to align datasets with different frequencies.

PYTHON CODE

```
ts = df.set_index('order_date')['revenue']
daily   = ts.resample('D').sum()
weekly  = ts.resample('W').sum()
monthly = ts.resample('M').sum()
print(monthly.head(12))
monthly.plot(figsize=(12,4), title='Monthly Revenue')
plt.show()
```

Resampling with .mean() vs .sum() gives different insights — use sum for totals (revenue), mean for rates (avg transaction value).

#058 Rolling Statistics

DEFINITION

Rolling (sliding window) statistics compute a metric over a moving window of fixed size. Common rolling stats: rolling mean (smooths noise), rolling standard deviation (tracks volatility), and rolling correlation. They reveal trends embedded in noisy time-series data and are used in finance (moving averages) and anomaly detection.

FORMULA

Rolling mean at t = (1/k) Σ■■■■■■■■ x■

WHEN TO USE

To smooth noisy time-series and identify underlying trends or changing volatility.

PYTHON CODE

```
monthly = df.resample('M', on='order_date')['revenue'].sum()
fig, ax = plt.subplots(figsize=(14, 5))
monthly.plot(ax=ax, alpha=0.4, label='Monthly')
monthly.rolling(3).mean().plot(ax=ax, label='3M Rolling Mean', lw=2)
monthly.rolling(6).mean().plot(ax=ax, label='6M Rolling Mean', lw=2)
ax.legend(); ax.set_title('Revenue with Rolling Averages')
plt.show()
```

When the 3M rolling mean crosses the 6M rolling mean from below, it often signals an upturn in the trend.

#059 Bivariate Analysis: Categorical vs Numerical

DEFINITION

Examining how a numeric variable's distribution differs across categories of a categorical variable. Techniques include box plots, violin plots, grouped histograms, and grouped summary tables. Statistical tests (ANOVA, t-test) determine if observed differences are statistically significant.

FORMULA

N/A

WHEN TO USE

To understand how categories influence a numeric outcome before modeling.

PYTHON CODE

```
# Summary statistics by group
print(df.groupby('category')['revenue'].agg(['mean','median','std','count']))
# Visualization
plt.figure(figsize=(12, 5))
sns.boxplot(data=df, x='order_status', y='revenue',
            order=df.groupby('order_status')['revenue'].median()                    .sort_values().index)
plt.xticks(rotation=20); plt.tight_layout(); plt.show()
```

If the inter-quartile ranges (box widths) differ significantly across groups, the variance is non-homogeneous — check this before ANOVA.

#060 Bivariate Analysis: Two Categorical Variables

DEFINITION

Analyzing the relationship between two categorical variables using cross-tabulation (frequency tables), stacked/grouped bar charts, and Chi-square tests. A cross-tab reveals the joint frequency distribution; normalizing by row or column gives conditional proportions.

FORMULA

N/A

WHEN TO USE

To understand the association between two categorical features.

PYTHON CODE

```
ct = pd.crosstab(df['gender'], df['category'],
                 normalize='index')  # row proportions
ct.plot(kind='bar', stacked=True, figsize=(8,5), colormap='Set2')
plt.title('Category Mix by Gender (Proportional)')
plt.ylabel('Proportion'); plt.xticks(rotation=0)
plt.legend(title='Category', bbox_to_anchor=(1.01,1))
plt.tight_layout(); plt.show()
```

Row-normalized cross-tab reveals category preferences by gender. Column-normalized shows gender composition per category.

#061 Cross-tabulation (pd.crosstab)

DEFINITION

pd.crosstab() creates a frequency matrix between two categorical variables. It supports multiple aggregation options: counts (default), margins (totals), normalization (proportions), and custom aggregation functions. The margins=True parameter adds row and column totals.

FORMULA

cell(r,c) = count(row_var=r AND col_var=c)

WHEN TO USE

Any time you need a joint frequency table between two categorical variables.

PYTHON CODE

```
ct = pd.crosstab(df['gender'], df['order_status'],
                 margins=True, margins_name='Total')
print(ct)
# Normalized (proportions)
ct_pct = pd.crosstab(df['gender'], df['order_status'],
                     normalize='all') * 100
print(ct_pct.round(1))
```

Margins reveal that 'Female' may have a higher 'Returned' rate than 'Male' — a business-relevant finding.

#062 Multivariate Analysis

DEFINITION

Multivariate analysis examines relationships among three or more variables simultaneously. Techniques include pair plots, 3D scatter plots, parallel coordinates plots, radar/spider charts, and dimensionality reduction (PCA). Multivariate insights often reveal patterns invisible in univariate or bivariate views.

FORMULA

N/A

WHEN TO USE

When bivariate analysis reveals complex interactions that require additional context.

PYTHON CODE

```
from pandas.plotting import parallel_coordinates
pc_df = df[['age','revenue','quantity','days_to_ship','gender']]          .dropna().sample(200)
pc_df['gender'] = pc_df['gender'].astype(str)
parallel_coordinates(pc_df, 'gender',
                     color=['steelblue','coral'], alpha=0.3)
plt.title('Parallel Coordinates Plot')
plt.show()
```

Lines that clearly separate by color in a parallel coordinates plot indicate that the variable on that axis strongly differentiates groups.

#063 Facet Grid / Subplot Grouping

DEFINITION

A facet grid creates a grid of subplots where each subplot shows the same visualization for a different subset of data (filtered by a row/column categorical variable). seaborn FacetGrid and relplot(col=, row=) enable this easily. It answers: 'Does this pattern hold across all subgroups?'

FORMULA

N/A

WHEN TO USE

When the same chart needs to be shown for multiple sub-groups for direct comparison.

PYTHON CODE

```
g = sns.FacetGrid(df.dropna(), col='gender', row='quarter',
                  height=3, aspect=1.5)
g.map(plt.hist, 'revenue', bins=20, color='steelblue',
      edgecolor='white')
g.set_axis_labels('Revenue', 'Count')
g.set_titles(col_template='Gender: {col_name}',
             row_template='Q{row_name}')
plt.show()
```

If the pattern in each facet looks identical, the variable used for faceting is not adding information. Use it when you expect heterogeneity.

#064 Formatting and Styling Plots

DEFINITION

Professional-quality visualizations require attention to: figure size, font sizes, axis labels, titles, color palettes, gridlines, tick formatting, and overall style. Matplotlib's rcParams and seaborn's set_theme() provide global style settings. Consistent formatting across all charts makes reports more readable.

FORMULA

N/A

WHEN TO USE

Before finalizing any visualization for reporting or presentation.

PYTHON CODE

```
plt.rcParams.update({
    'figure.dpi': 100,
    'font.family': 'DejaVu Sans',
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.grid': True,
    'grid.alpha': 0.3
})
sns.set_theme(style='whitegrid', palette='Set2', font_scale=1.1)
```

Removing top and right spines (axes.spines) creates a cleaner, more modern look. Always include axis labels and a descriptive title.

#065 Adding Annotations to Plots

DEFINITION

Annotations add text labels, arrows, or shapes to highlight specific data points or regions in a chart. plt.annotate() places text with an optional arrow; plt.axhline()/axvline() adds reference lines; plt.text() adds free text. Annotations are critical for communicating key findings directly on the chart.

FORMULA

N/A

WHEN TO USE

To highlight outliers, thresholds, notable events (e.g., a policy change date), or peak values.

PYTHON CODE

```
fig, ax = plt.subplots(figsize=(12, 5))
monthly.plot(ax=ax)
peak_month = monthly.idxmax()
peak_val = monthly.max()
ax.annotate(f'Peak: {peak_val:,.0f}',
            xy=(peak_month, peak_val),
            xytext=(peak_month, peak_val * 0.85),
            arrowprops=dict(arrowstyle='->', color='red'),
            fontsize=10, color='red')
ax.axhline(monthly.mean(), ls='--', color='gray', label='Mean')
ax.legend(); plt.show()
```

Good annotations transform a chart from descriptive to analytical — they guide the viewer to the most important insight.

#066 Exporting Plots

DEFINITION

Saving plots to files for use in reports, presentations, or dashboards. plt.savefig() supports formats: PNG (raster, for web), PDF/SVG (vector, for print), and JPEG. Key parameters: dpi (resolution), bbox_inches='tight' (prevents label clipping), and transparent=True (for overlaying on colored backgrounds).

FORMULA

N/A

WHEN TO USE

Whenever plots need to be included in external documents or dashboards.

PYTHON CODE

```
fig.savefig('revenue_trend.png', dpi=150, bbox_inches='tight',
            facecolor='white')
fig.savefig('revenue_trend.pdf', bbox_inches='tight')
# Or using a context manager:
with plt.style.context('seaborn-v0_8-whitegrid'):
    fig, ax = plt.subplots()
    # ...plotting...
    fig.savefig('styled_chart.png', dpi=150, bbox_inches='tight')
```

Use dpi=150 for reports; dpi=300 for publication quality. Vector formats (SVG/PDF) scale without loss — preferred for presentations.

#067 Insights Documentation

DEFINITION

EDA findings should be documented in structured markdown cells or a summary report. For each visualization, record: what was plotted, the key observation, and its business implication. This turns raw charts into actionable intelligence and forms the evidence base for recommendations.

FORMULA

N/A

WHEN TO USE

After each EDA visualization — don't just make charts, interpret them.

PYTHON CODE

```
# Example documentation structure:
insights = {
    'revenue_distribution': {
        'observation': 'Revenue is right-skewed (skew=2.3) with median=142 vs mean=189',
        'implication': 'A small number of high-value orders drives average up. Report median not mean.',
        'recommended_action': 'Investigate top 5% orders for VIP program targeting'
    }
}
for key, val in insights.items():
    print(f'\n{key}:')
    for k, v in val.items():
        print(f'  {k}: {v}')
```

Insights should follow the format: OBSERVATION + WHY IT MATTERS + RECOMMENDED ACTION.

#068 Missing Pattern Visualization

DEFINITION

Beyond simple missing counts, visualizing the spatial pattern of missing values reveals whether missingness is random or systematic. The missingno library provides matrix plots (row-level pattern), bar charts (column-level summary), and heatmaps (column co-occurrence of missing values). Systematic patterns suggest data collection failures.

FORMULA

N/A

WHEN TO USE

When you have >5 columns with missing values and want to understand if they are related.

PYTHON CODE

```
import missingno as msno
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
# Missing pattern matrix
msno.matrix(df, ax=axes[0], sparkline=False, color=(0.27, 0.52, 0.71))
axes[0].set_title('Missing Value Matrix')
# Bar chart of completeness
msno.bar(df, ax=axes[1], color='steelblue')
axes[1].set_title('Column Completeness')
plt.tight_layout(); plt.show()
```

Horizontal bands of white in the matrix = entire rows missing (network/system outage). Vertical white stripes = entire column missing (sensor failure).

#069 Distribution Comparison Across Groups

DEFINITION

Systematically comparing a numeric distribution between multiple sub-groups using overlapping KDE plots, side-by-side histograms, or a grid of box plots. The goal is to identify which groups have different central tendencies, spread, or shapes — forming hypotheses for formal statistical testing.

FORMULA

N/A

WHEN TO USE

Before running group comparison tests (t-test, ANOVA) to visually check assumptions.

PYTHON CODE

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
# Overlapping KDE
for cat in df['category'].unique():
    df[df['category']==cat]['revenue'].plot.kde(ax=axes[0], label=cat)
axes[0].set_title('Revenue KDE by Category'); axes[0].legend()
# Box plot comparison
df.boxplot(column='revenue', by='category', ax=axes[1], rot=30)
axes[1].set_title('Revenue Distribution by Category')
plt.suptitle(''); plt.tight_layout(); plt.show()
```

If KDE peaks are in very different locations across groups, a statistical test will likely confirm significance.

#070 Outlier Visualization

DEFINITION

Dedicated visualizations for outlier identification go beyond box plots. Scatter plots of individual values vs index, strip plots, and 2D scatter plots colored by outlier status help identify where and why outliers occur. Context-aware outlier visualization (e.g., outliers by date) can reveal data entry patterns.

FORMULA

N/A

WHEN TO USE

After outlier detection, to visually validate detected outliers and understand their context.

PYTHON CODE

```
Q1, Q3 = df['revenue'].quantile([0.25, 0.75])
IQR = Q3 - Q1
is_outlier = (df['revenue'] < Q1 - 1.5*IQR) | (df['revenue'] > Q3 + 1.5*IQR)
plt.figure(figsize=(12, 5))
plt.scatter(range(len(df)), df['revenue'],
            c=is_outlier.map({True:'red', False:'steelblue'}),
            alpha=0.4, s=15)
plt.title(f'Revenue Outliers ({is_outlier.sum()} flagged in red)')
plt.xlabel('Row Index'); plt.ylabel('Revenue')
plt.show()
```

Outliers clustered in early/late rows suggest a data collection period with quality issues, not random errors.

Chapter 3

Statistical Analysis

Tests, correlations, regression

46 Concept

#071 Normal Distribution

DEFINITION

The normal (Gaussian) distribution is a continuous probability distribution defined by mean (μ) and standard deviation (σ). It is symmetric and bell-shaped, with 68% of data within ±1σ, 95% within ±2σ, and 99.7% within ±3σ (the empirical rule). Many statistical tests assume normality, making it the most important distribution in statistics.

FORMULA

$f(x) = (1/\sigma\sqrt{(2\pi)}) \cdot \exp(-(x-\mu)^2/2\sigma^2)$

μ = mean, σ = standard deviation

WHEN TO USE

As a reference distribution for parametric tests and understanding data spread.

PYTHON CODE

```
from scipy.stats import norm
import numpy as np

mu, sigma = df['revenue'].mean(), df['revenue'].std()
x = np.linspace(mu - 4*sigma, mu + 4*sigma, 200)
plt.figure(figsize=(10, 5))
plt.hist(df['revenue'], bins=40, density=True, alpha=0.6, label='Data')
plt.plot(x, norm.pdf(x, mu, sigma), 'r-', lw=2, label='Normal fit')
plt.title(f'Revenue vs Normal Distribution (μ={mu:.0f}, σ={sigma:.0f})')
plt.legend(); plt.show()
```

If the histogram deviates significantly from the normal curve (especially in tails), parametric tests may be unreliable.

#072 Standard Normal Distribution (Z-distribution)

DEFINITION

The standard normal distribution is a special case of the normal distribution with μ=0 and σ=1. Any normal variable X can be standardized to Z = (X–μ)/σ, enabling the use of standard normal tables for probability calculations. Z-scores measure how many standard deviations an observation is from the mean.

FORMULA

Z = (X – μ) / σ

Z ~ N(0, 1)

WHEN TO USE

For standardizing variables, computing probabilities, and comparing values across different scales.

PYTHON CODE

```
from scipy.stats import norm

# Probability P(X > 250) for revenue
mu, sigma = df['revenue'].mean(), df['revenue'].std()
z = (250 - mu) / sigma
p_above = 1 - norm.cdf(z)
print(f'P(Revenue > 250) = {p_above:.4f} ({p_above*100:.2f}%)')

# Standardize a column
df['revenue_z'] = (df['revenue'] - mu) / sigma
```

A revenue_z of +2 means that customer's revenue is 2 standard deviations above average — in the top ~2.3%.

#073 Shapiro-Wilk Test

DEFINITION

The Shapiro-Wilk test is the most powerful normality test for small to moderate samples (n < 2000). The null hypothesis is that the data is normally distributed. A significant p-value (p < 0.05) rejects normality. The test statistic W ranges from 0 to 1 — values close to 1 indicate normality.

FORMULA

W = (Σ a■ x(■))² / Σ(x■ – x■)²

H■: data is normally distributed

WHEN TO USE

Before applying parametric tests (t-test, ANOVA) to verify the normality assumption.

PYTHON CODE

```
from scipy.stats import shapiro

for col in ['age', 'revenue', 'quantity']:
    sample = df[col].dropna().sample(min(1000, len(df)), random_state=42)
    stat, p = shapiro(sample)
    result = 'Normal' if p > 0.05 else 'NOT Normal'
    print(f'{col}: W={stat:.4f}, p={p:.4f} → {result}')
```

For large datasets (n>2000), even tiny deviations from normality yield p<0.05. Always combine with Q-Q plots for visual assessment.

#074 D'Agostino-Pearson Test

DEFINITION

The D'Agostino-Pearson K² test combines skewness and kurtosis tests into a single omnibus normality test. It works well for larger samples where Shapiro-Wilk loses power. scipy.stats.normaltest() implements this. Like Shapiro-Wilk, p < 0.05 rejects normality.

FORMULA

$K^2 = Z^2\_skewness + Z^2\_kurtosis \sim \chi^2(2)$ under H■

WHEN TO USE

As an alternative or complement to Shapiro-Wilk, especially for n > 2000.

PYTHON CODE

```
from scipy.stats import normaltest

for col in ['age', 'revenue', 'quantity']:
    clean = df[col].dropna()
    stat, p = normaltest(clean)
    result = 'Normal' if p > 0.05 else 'NOT Normal'
    print(f'{col}: K²={stat:.4f}, p={p:.6f} → {result}')
```

■ ***When both Shapiro-Wilk and D'Agostino agree (both reject), non-normality is well-established. When they disagree, inspect the Q-Q plot.***

#075 Kolmogorov-Smirnov Test

DEFINITION

The Kolmogorov-Smirnov (K-S) test measures the maximum difference (D statistic) between the empirical cumulative distribution function (CDF) of the sample and the theoretical CDF of a specified distribution (e.g., normal). A one-sample test checks fit to a distribution; a two-sample test compares two empirical distributions.

FORMULA

$D = \sup|F\_n(x) - F(x)|$

H■: sample follows the specified distribution

WHEN TO USE

For testing fit to any distribution (not just normal), or comparing two empirical distributions.

PYTHON CODE

```
from scipy.stats import kstest, norm

col = 'revenue'
data = df[col].dropna()
# Normalize
z_data = (data - data.mean()) / data.std()
stat, p = kstest(z_data, 'norm')
print(f'K-S test: D={stat:.4f}, p={p:.4f}')
print('Result:', 'Normal' if p > 0.05 else 'NOT Normal')
```

■ ***K-S is less powerful than Shapiro-Wilk for detecting normality departures but is more general (can test any distribution).***

#076 Q-Q Plot (Quantile-Quantile)

DEFINITION

A Q-Q (Quantile-Quantile) plot compares the quantiles of the sample data against the quantiles of a theoretical distribution (usually normal). If the data follows the theoretical distribution, points fall along a straight diagonal reference line. Deviations from the line reveal systematic departures: S-curve = skewness, heavy tails = kurtosis.

FORMULA

Plot: (theoretical quantile, sample quantile) pairs

WHEN TO USE

For visual normality assessment — always pair with a formal test.

PYTHON CODE

```
import scipy.stats as stats
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for ax, col in zip(axes, ['age', 'revenue', 'quantity']):
    stats.probplot(df[col].dropna(), dist='norm', plot=ax)
    ax.set_title(f'Q-Q Plot: {col}')
    ax.get_lines()[1].set_color('red')
plt.tight_layout(); plt.show()
```

Points bowing upward at ends = right tail heavier than normal (positive kurtosis). S-shaped deviation = skewness.

#077 Central Limit Theorem (CLT)

DEFINITION

The Central Limit Theorem states that the distribution of sample means approaches a normal distribution as sample size increases, regardless of the population's distribution shape. For n ≥ 30, the sampling distribution of the mean is approximately normal. This is why many parametric tests (t-tests, z-tests) work even on non-normal data when samples are large.

FORMULA

$X\blacksquare \sim N(\mu, \sigma^2/n)$ as $n \to \infty$

$SE = \sigma/\sqrt{n}$ (standard error of the mean)

WHEN TO USE

To justify using normal-based tests on non-normal data when sample sizes are large.

PYTHON CODE

```
# Demonstrate CLT
np.random.seed(42)
population = df['revenue'].dropna().values
sample_means = [np.random.choice(population, 50).mean() for _ in range(2000)]
plt.figure(figsize=(10, 4))
plt.hist(sample_means, bins=40, edgecolor='white', color='steelblue')
plt.title(f'Distribution of 2000 Sample Means (n=50)\nLooks Normal even though Revenue is Skewed')
plt.show()
```

Even with highly skewed revenue, sample means of n=50 are already approximately normally distributed — validating t-tests.

#078 Standard Error of the Mean

DEFINITION

The Standard Error (SE) quantifies the variability of sample means across repeated samples. It is the standard deviation divided by the square root of sample size: SE = σ/√n. A smaller SE means the sample mean is a more precise estimate of the population mean. SE shrinks as sample size increases.

FORMULA

SE = σ/√n (population SE)
SE = s/√n (sample SE, using sample std)

WHEN TO USE

When quantifying precision of a mean estimate or constructing confidence intervals.

PYTHON CODE

```
from scipy.stats import sem

for col in ['age', 'revenue', 'quantity']:
    clean = df[col].dropna()
    mean_val = clean.mean()
    se_val = sem(clean)
    n = len(clean)
    print(f'{col}: mean={mean_val:.2f}, SE={se_val:.4f}, n={n}')
```

SE tells you how much you'd expect the sample mean to vary if you drew another sample of the same size.

#079 Confidence Interval for the Mean

DEFINITION

A 95% confidence interval (CI) means that if we repeated the sampling process 100 times, 95 of the resulting intervals would contain the true population mean. It is constructed as: mean ± t* × SE, where t* is the t-distribution critical value for the desired confidence level and degrees of freedom.

FORMULA

CI = x■ ± t*(α/2, n–1) × (s/√n)
t*(0.025, n–1) ≈ 1.96 for large n

WHEN TO USE

To express the precision and uncertainty of a sample mean estimate.

PYTHON CODE

```
from scipy import stats

for col in ['revenue', 'age']:
    clean = df[col].dropna()
    ci = stats.t.interval(confidence=0.95,
                          df=len(clean)-1,
                          loc=clean.mean(),
                          scale=stats.sem(clean))
    print(f'{col}: 95% CI = ({ci[0]:.2f}, {ci[1]:.2f})')
```

A narrower CI indicates a more precise estimate. If CIs for two groups don't overlap, the means are likely significantly different.

#080 Confidence Interval for a Proportion

DEFINITION

For proportions (binary outcomes), the CI is built using the normal approximation to the binomial. The Wilson score interval is more accurate for extreme proportions (near 0 or 1) or small samples. A 95% CI for a proportion p is: p■ ± 1.96 × √(p■(1–p■)/n).

FORMULA

CI = p■ ± z_(α/2) × √(p■(1–p■)/n)
Wilson score is more accurate for small n

WHEN TO USE

When estimating uncertainty around a proportion (e.g., return rate, conversion rate).

PYTHON CODE

```
from statsmodels.stats.proportion import proportion_confint

# Return rate CI
returns = (df['order_status'] == 'Returned').sum()
n_total = df['order_status'].notna().sum()
ci_low, ci_high = proportion_confint(returns, n_total,
                                     alpha=0.05, method='wilson')
p_hat = returns / n_total
print(f'Return rate: {p_hat:.3f} (Wilson 95% CI: [{ci_low:.3f}, {ci_high:.3f}])')
```

■ ***The Wilson interval is preferred when p < 0.1 or n < 100. The standard normal approximation becomes unreliable in these cases.***

#081 Bootstrapped Confidence Interval

DEFINITION

Bootstrapping estimates the sampling distribution of any statistic by resampling with replacement from the observed data thousands of times. The 2.5th and 97.5th percentiles of the bootstrap distribution form the 95% CI. It requires no distributional assumptions and works for any statistic (mean, median, correlation, etc.).

FORMULA

CI = [Q(2.5%) of bootstrap dist., Q(97.5%) of bootstrap dist.]
Bootstrap: resample n observations with replacement B times

WHEN TO USE

For non-normal data, small samples, or statistics without closed-form CIs (e.g., median, Gini coefficient).

PYTHON CODE

```
np.random.seed(42)
data = df['revenue'].dropna().values
B = 5000
boot_means = [np.random.choice(data, len(data), replace=True).mean()
              for _ in range(B)]
ci_low, ci_high = np.percentile(boot_means, [2.5, 97.5])
print(f'Bootstrap 95% CI for mean revenue: [{ci_low:.2f}, {ci_high:.2f}]')
```

■ ***Bootstrap CIs can be asymmetric — this is a feature, not a bug, reflecting the true asymmetry in the sampling distribution.***

#082 Hypothesis Testing Framework

DEFINITION

Hypothesis testing is a structured procedure for making statistical decisions. Steps: (1) State H■ (null) and H■ (alternative) hypotheses. (2) Choose significance level α (usually 0.05). (3) Compute test statistic and p-value. (4) Decision: if p < α, reject H■. The p-value is the probability of observing results at least as extreme as observed, assuming H■ is true.

FORMULA

p-value = P(test statistic ≥ observed | H■ is true)
Reject H■ if p-value < α

WHEN TO USE

Whenever you want to make an evidence-based statistical claim about a population.

PYTHON CODE

```
# Template for any hypothesis test
def report_test(test_name, stat, p, alpha=0.05):
    print(f'\n{test_name}')
    print(f'  Test statistic: {stat:.4f}')
    print(f'  p-value: {p:.4f}')
    if p < alpha:
        print(f'  → Reject H■ (p={p:.4f} < α={alpha}): Significant')
    else:
        print(f'  → Fail to reject H■ (p={p:.4f} ≥ α={alpha}): Not Significant')
```

■ ***p < 0.05 does NOT mean the effect is large or important. Always report effect size alongside p-values.***

#083 Type I and Type II Errors

DEFINITION

Type I error (false positive, α): Rejecting H■ when it is actually true. The probability of a Type I error is α (the significance level). Type II error (false negative, β): Failing to reject H■ when it is actually false. Statistical power = 1 – β. There is a trade-off: reducing α increases β. A power of 0.80 (β=0.20) is the conventional minimum for study design.

FORMULA

α = P(Type I error) = P(reject H■ | H■ true)
β = P(Type II error) = P(fail to reject H■ | H■ true)
Power = 1 – β

WHEN TO USE

When designing studies (sample size calculation) and interpreting test results.

PYTHON CODE

```
from statsmodels.stats.power import TTestIndPower

# How many samples needed for 80% power?
analysis = TTestIndPower()
n = analysis.solve_power(effect_size=0.5,  # Cohen's d
                         power=0.8,
                         alpha=0.05,
                         alternative='two-sided')
print(f'Required sample size per group: {n:.0f}')
```

■ ***With small samples, underpowered tests often miss real effects (Type II errors). Don't interpret non-significant results as 'no effect'.***

#084 One-Sample t-Test

DEFINITION

Tests whether the mean of a single sample is significantly different from a specified hypothesized value (μ■). For example: 'Is the average revenue different from $150?' Uses the t-distribution with n–1 degrees of freedom. Appropriate when population variance is unknown (which is almost always in practice).

FORMULA

t = (x■ – μ■) / (s/√n)
H■: μ = μ■ vs H■: μ ≠ μ■

WHEN TO USE

To compare a sample mean to a known benchmark or target value.

PYTHON CODE

```
from scipy.stats import ttest_1samp

mu_0 = 150  # Hypothesized mean revenue
stat, p = ttest_1samp(df['revenue'].dropna(), popmean=mu_0)
print(f'One-Sample t-test: t={stat:.4f}, p={p:.4f}')
print(f'Sample mean: {df.revenue.mean():.2f}')
if p < 0.05:
    print('Reject H■: Mean revenue significantly differs from 150')
```

If p < 0.05, the sample mean is statistically significantly different from μ■. Check CI to see the direction and magnitude.

#085 Two-Sample Independent t-Test (Welch's)

DEFINITION

Tests whether the means of two independent groups are significantly different. Welch's t-test (equal_var=False) does not assume equal variances between groups, making it more robust than the classic Student's t-test. It adjusts the degrees of freedom using the Welch-Satterthwaite equation.

FORMULA

t = (x■■ – x■■) / √(s■²/n■ + s■²/n■)
degrees of freedom: Welch-Satterthwaite approximation

WHEN TO USE

To compare means of two independent groups (e.g., male vs female average revenue).

PYTHON CODE

```
from scipy.stats import ttest_ind

male_rev = df[df['gender']=='Male']['revenue'].dropna()
female_rev = df[df['gender']=='Female']['revenue'].dropna()
stat, p = ttest_ind(male_rev, female_rev, equal_var=False)
print(f"Male mean: {male_rev.mean():.2f}, Female mean: {female_rev.mean():.2f}")
print(f'Welch t-test: t={stat:.4f}, p={p:.4f}')
```

Always use equal_var=False (Welch's) unless you have strong evidence of equal variances. It is the safer default.

#086 Paired t-Test

DEFINITION

Tests whether the mean difference between paired observations (same subject measured twice) is significantly different from zero. Used for before/after studies, matched-pair designs, or repeated measurements. Reduces individual variability by focusing on within-subject changes.

FORMULA

t = d■ / (s_d/√n)

d = difference within each pair, d■ = mean of differences

WHEN TO USE

For before/after comparisons on the same subjects (e.g., revenue before vs after a promotion).

PYTHON CODE

```
from scipy.stats import ttest_rel

# Example: Comparing two price periods
# (Simulated: first half vs second half for same customers)
group1 = df.iloc[:500]['revenue'].dropna().reset_index(drop=True)
group2 = df.iloc[500:1000]['revenue'].dropna().reset_index(drop=True)
n = min(len(group1), len(group2))
stat, p = ttest_rel(group1[:n], group2[:n])
print(f'Paired t-test: t={stat:.4f}, p={p:.4f}')
```

■ ***If the same customer's revenue is being compared before/after a campaign, use paired t-test. For different customers, use independent t-test.***

#087 Mann-Whitney U Test

DEFINITION

The Mann-Whitney U test is the non-parametric equivalent of the independent two-sample t-test. It tests whether one group's values tend to be systematically larger than the other's. It compares ranks, not means, and makes no normality assumption. The null hypothesis is that both groups come from the same distribution.

FORMULA

U = n■n■ + n■(n■+1)/2 – R■

R■ = sum of ranks for group 1

WHEN TO USE

When normality is violated and sample sizes are small, or when comparing medians is more appropriate than means.

PYTHON CODE

```
from scipy.stats import mannwhitneyu

male_rev = df[df['gender']=='Male']['revenue'].dropna()
female_rev = df[df['gender']=='Female']['revenue'].dropna()
stat, p = mannwhitneyu(male_rev, female_rev, alternative='two-sided')
print(f'Mann-Whitney U: U={stat:.0f}, p={p:.4f}')
print(f'Male median: {male_rev.median():.2f}')
print(f'Female median: {female_rev.median():.2f}')
```

■ ***Use Mann-Whitney when revenue is skewed or when comparing medians is the research question. It is more robust than t-test for non-normal data.***

#088 Levene's Test for Equal Variances

DEFINITION

Levene's test determines whether two or more groups have equal variances (homoscedasticity). It is the recommended pre-test before ANOVA (which assumes homogeneity of variance). Unlike Bartlett's test, Levene's is robust to non-normality. H■: all group variances are equal; p < 0.05 rejects equal variances.

FORMULA

W = ((N-k)/(k-1)) × Σn■(Z■.-Z..)² / ΣΣ(Z■■-Z■.)²
Z■■ = |X■■ - x■■| (deviation from group median)

WHEN TO USE

Before ANOVA and two-sample t-tests, to check variance homogeneity.

PYTHON CODE

```
from scipy.stats import levene

groups = [df[df['category']==cat]['revenue'].dropna()
          for cat in df['category'].unique()]
stat, p = levene(*groups)
print(f"Levene's test: W={stat:.4f}, p={p:.4f}")
if p < 0.05:
    print('Groups have UNEQUAL variances → use Welch ANOVA or Kruskal-Wallis')
else:
    print('Groups have EQUAL variances → standard ANOVA is valid')
```

If Levene's rejects equal variances (p < 0.05), use Welch's one-way ANOVA (welch_anova in pingouin) or the non-parametric Kruskal-Wallis.

#089 F-Test for Variance

DEFINITION

The F-test compares the variances of two populations. F = s■²/s■², and the test statistic follows an F-distribution with (n■–1) and (n■–1) degrees of freedom. In regression analysis, the overall F-test tests whether the regression model explains significantly more variance than the null model (intercept-only).

FORMULA

F = s■² / s■²
H■: σ■² = σ■² (in variance comparison)
F = MSR/MSE (in ANOVA/regression)

WHEN TO USE

To compare two sample variances or to assess overall regression model significance.

PYTHON CODE

```
from scipy.stats import f

s1 = df[df['gender']=='Male']['revenue'].std()
s2 = df[df['gender']=='Female']['revenue'].std()
n1 = df[df['gender']=='Male']['revenue'].notna().sum()
n2 = df[df['gender']=='Female']['revenue'].notna().sum()
F = s1**2 / s2**2
p = 2 * min(f.cdf(F, n1-1, n2-1), 1-f.cdf(F, n1-1, n2-1))
print(f'F = {F:.4f}, p = {p:.4f}')
```

F ≈ 1 means variances are equal. F >> 1 means group 1 has larger variance. Check if this matters for your analysis assumptions.

#090 One-Way ANOVA

DEFINITION

Analysis of Variance (ANOVA) tests whether the means of three or more independent groups are significantly different. It partitions total variance into between-group variance (SSB) and within-group variance (SSW). The F-statistic = MSB/MSW. H■: all group means are equal. A significant result only tells you that at least one pair differs — post-hoc tests identify which pairs.

FORMULA

F = MSB/MSW = [SSB/(k–1)] / [SSW/(N–k)]
H■: μ■ = μ■ = ... = μ■

WHEN TO USE

To compare means across 3+ groups when normality and homogeneity of variance hold.

PYTHON CODE

```
from scipy.stats import f_oneway

groups = [df[df['category']==cat]['revenue'].dropna()
          for cat in df['category'].unique()]
stat, p = f_oneway(*groups)
print(f'One-Way ANOVA: F={stat:.4f}, p={p:.4f}')
if p < 0.05:
    print('At least one category has a different mean revenue → run post-hoc test')
```

■ ***ANOVA tells you IF groups differ but not WHICH groups. Always follow a significant ANOVA with Tukey HSD or another post-hoc test.***

#091 Tukey HSD Post-Hoc Test

DEFINITION

Tukey's Honest Significant Difference (HSD) test performs all pairwise mean comparisons after a significant ANOVA, while controlling the family-wise error rate. It tests every pair of groups and provides adjusted p-values and confidence intervals for each pairwise mean difference. It is the most commonly used post-hoc test for balanced designs.

FORMULA

q = (x■■ – x■■) / √(MSW/n■)
Critical value from Studentized Range Distribution

WHEN TO USE

After a significant one-way ANOVA to identify which specific group pairs differ.

PYTHON CODE

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd

tukey = pairwise_tukeyhsd(endog=df['revenue'].dropna(),
                          groups=df.loc[df['revenue'].notna(), 'category'],
                          alpha=0.05)
print(tukey.summary())
tukey.plot_simultaneous(figsize=(8, 5))
plt.title('Tukey HSD — 95% CIs for Mean Differences')
plt.show()
```

■ ***If the CI for a group pair does not include 0, those two groups differ significantly. The 'reject' column confirms this.***

#092 Kruskal-Wallis Test

DEFINITION

The Kruskal-Wallis test is the non-parametric equivalent of one-way ANOVA. It ranks all observations, then tests whether the average rank differs across groups. It makes no normality or homoscedasticity assumptions. H■: all groups have the same distribution (equivalent medians). Like ANOVA, it needs post-hoc tests to identify which groups differ.

FORMULA

H = (12/N(N+1)) Σ (R■²/n■) – 3(N+1)
H ~ χ²(k–1) under H■

WHEN TO USE

When ANOVA assumptions are violated (non-normal data or unequal variances across groups).

PYTHON CODE

```
from scipy.stats import kruskal

groups = [df[df['category']==cat]['revenue'].dropna()
          for cat in df['category'].unique()]
stat, p = kruskal(*groups)
print(f'Kruskal-Wallis: H={stat:.4f}, p={p:.4f}')
# Post-hoc: Dunn test
from scikit_posthocs import posthoc_dunn
posthoc = posthoc_dunn(df, val_col='revenue', group_col='category', p_adjust='bonferroni')
print(posthoc)
```

■ ***Kruskal-Wallis compares medians (not means). Use it as default when normality is questionable; use ANOVA only when normality is confirmed.***

#093 Two-Way ANOVA

DEFINITION

Two-Way ANOVA tests the effect of two categorical independent variables (factors) on a continuous dependent variable, including their interaction effect. Factor A main effect: does mean vary by factor A? Factor B main effect: does mean vary by factor B? Interaction (A×B): does the effect of A depend on the level of B?

FORMULA

SS_Total = SS_A + SS_B + SS_AB + SS_Error
F_A = MS_A/MS_Error, F_B = MS_B/MS_Error,
F_AB = MS_AB/MS_Error

WHEN TO USE

When two categorical variables both potentially influence the outcome and you want to test both main effects and their interaction.

PYTHON CODE

```
import pingouin as pg

result = pg.anova(data=df.dropna(subset=['revenue','gender','category']),
                  dv='revenue',
                  between=['gender', 'category'],
                  detailed=True)
print(result[['Source','SS','ddof1','F','p-unc','np2']].round(4))
```

■ ***If the interaction term is significant (p < 0.05), the effect of gender on revenue differs by category — don't interpret main effects in isolation.***

#094 Effect Size — Cohen's d

DEFINITION

Effect size quantifies the practical magnitude of a difference, independent of sample size. Cohen's d measures the standardized difference between two group means: d = (μ■–μ■) / σ_pooled. Guidelines: small = 0.2, medium = 0.5, large = 0.8. A large sample can make a tiny difference statistically significant — effect size tells you if it's practically meaningful.

FORMULA

d = (x■■ – x■■) / s_pooled
s_pooled = √((s■² + s■²) / 2)

WHEN TO USE

Always report alongside t-test or ANOVA results to convey practical significance.

PYTHON CODE

```
def cohens_d(g1, g2):
    n1, n2 = len(g1), len(g2)
    s_pool = np.sqrt(((n1-1)*g1.std()**2 + (n2-1)*g2.std()**2) / (n1+n2-2))
    return (g1.mean() - g2.mean()) / s_pool

male_rev = df[df['gender']=='Male']['revenue'].dropna()
female_rev = df[df['gender']=='Female']['revenue'].dropna()
d = cohens_d(male_rev, female_rev)
print(f"Cohen's d = {d:.4f}")
print(f"Effect size: {'small' if abs(d)<0.2 else 'medium' if abs(d)<0.5 else 'large'}")
```

■ ***d < 0.2 = trivial effect. A statistically significant result with d=0.05 means the groups barely differ in practice.***

#095 Statistical Power

DEFINITION

Statistical power (1–β) is the probability of correctly detecting a true effect when one exists. Power depends on: effect size (larger effect = higher power), sample size (more data = higher power), and α level. Most studies aim for power ≥ 0.80 (80%). Power analysis before a study determines the minimum sample size needed to detect an effect of a given size.

FORMULA

Power = 1 – β = P(reject H■ | H■ is true)
Sample size ↑ → Power ↑

WHEN TO USE

Before collecting data (a priori) or when interpreting non-significant results.

PYTHON CODE

```
from statsmodels.stats.power import TTestIndPower

pwr = TTestIndPower()
# Given observed effect, what was our power?
male_rev = df[df['gender']=='Male']['revenue'].dropna()
female_rev = df[df['gender']=='Female']['revenue'].dropna()
from scipy.stats import ttest_ind
_, p_obs = ttest_ind(male_rev, female_rev, equal_var=False)
d_obs = abs(male_rev.mean()-female_rev.mean())/np.sqrt((male_rev.std()**2+female_rev.std()**2)/2)
pwr_obs = pwr.power(effect_size=d_obs, nobs1=len(male_rev), alpha=0.05)
print(f'Observed power: {pwr_obs:.3f}')
```

■ ***Power < 0.80 means there is >20% chance of missing a real effect. Non-significant results from underpowered studies cannot be interpreted as 'no effect'.***

#096 Pearson Correlation

DEFINITION

The Pearson correlation coefficient (r) measures the strength and direction of the linear relationship between two continuous variables. r = +1 means perfect positive linear relationship; r = –1 means perfect negative; r = 0 means no linear relationship. It assumes both variables are normally distributed and the relationship is linear.

FORMULA

r = Σ((x■–x■)(y■–■)) / √(Σ(x■–x■)² × Σ(y■–■)²)

WHEN TO USE

For measuring linear association between two continuous, approximately normal variables.

PYTHON CODE

```
from scipy.stats import pearsonr

pairs = [('age','revenue'), ('quantity','revenue'), ('unit_price','revenue')]
for x_col, y_col in pairs:
    clean = df[[x_col, y_col]].dropna()
    r, p = pearsonr(clean[x_col], clean[y_col])
    print(f'r({x_col}, {y_col}) = {r:.4f}, p = {p:.4f}')
```

r > 0.7 = strong positive linear. r² = coefficient of determination = % variance in y explained by x.

#097 Spearman Correlation

DEFINITION

The Spearman rank correlation (ρ) measures the monotonic (not necessarily linear) relationship between two variables. It works on the ranks of the data rather than raw values, making it robust to outliers and non-normal distributions. ρ = +1 means perfect monotone increasing; ρ = –1 means perfect monotone decreasing.

FORMULA

ρ = 1 – (6 Σd■²) / (n(n²–1))

d■ = difference in paired ranks

WHEN TO USE

When variables are not normally distributed, contain outliers, or the relationship may be monotone but not linear.

PYTHON CODE

```
from scipy.stats import spearmanr

pairs = [('age','revenue'), ('quantity','revenue')]
for x_col, y_col in pairs:
    clean = df[[x_col, y_col]].dropna()
    rho, p = spearmanr(clean[x_col], clean[y_col])
    print(f'Spearman ρ({x_col}, {y_col}) = {rho:.4f}, p = {p:.4f}')
```

When Pearson r and Spearman ρ differ substantially, the relationship is monotone but non-linear, or outliers are influencing Pearson.

#098 Kendall's Tau

DEFINITION

Kendall's tau (τ) measures ordinal association between two variables by comparing concordant and discordant pairs. A concordant pair: both variables increase together; discordant: one increases while the other decreases. τ is less sensitive to outliers than both Pearson and Spearman and is preferred for small samples or ordinal data with many ties.

FORMULA

τ = (C – D) / √((C+D+T■)(C+D+T■))

C = concordant pairs, D = discordant pairs

WHEN TO USE

For ordinal data, ranked data, or small samples with many tied values.

PYTHON CODE

```
from scipy.stats import kendalltau

clean = df[['age','revenue']].dropna()
tau, p = kendalltau(clean['age'], clean['revenue'])
print(f"Kendall's τ = {tau:.4f}, p = {p:.4f}")
```

Kendall's τ is more interpretable than Spearman ρ: τ=0.3 means 30% more concordant pairs than discordant pairs.

#099 Point-Biserial Correlation

DEFINITION

The Point-Biserial correlation measures the relationship between one continuous variable and one dichotomous (binary) variable. It is mathematically equivalent to the Pearson correlation applied to this mixed case. Values range from –1 to +1, and the direction indicates which binary group has higher values.

FORMULA

r_pb = (M■ – M■) / s_total × √(n■n■/n²)

WHEN TO USE

When correlating a continuous variable with a binary categorical variable (e.g., revenue and is_returned flag).

PYTHON CODE

```
from scipy.stats import pointbiserialr

df['is_returned'] = (df['order_status'] == 'Returned').astype(int)
clean = df[['revenue','is_returned']].dropna()
r, p = pointbiserialr(clean['is_returned'], clean['revenue'])
print(f'Point-biserial r = {r:.4f}, p = {p:.4f}')
print(f'Mean revenue - Returned: {df[df.is_returned==1].revenue.mean():.2f}')
print(f'Mean revenue - Not Returned: {df[df.is_returned==0].revenue.mean():.2f}')
```

Negative r means returned orders have lower revenue. The sign tells you which group tends to score higher.

#100 Partial Correlation

DEFINITION

Partial correlation measures the relationship between two variables while controlling for the effect of one or more other variables. This removes the confounding influence of third variables. For example, the correlation between age and revenue after removing the effect of quantity.

FORMULA

$r_{xy.z} = (r_{xy} - r_{xz} \cdot r_{yz}) / \sqrt{((1-r_{xz}^2)(1-r_{yz}^2))}$

WHEN TO USE

When you suspect a third variable is confounding the observed correlation between two variables.

PYTHON CODE

```
import pingouin as pg

result = pg.partial_corr(data=df[['age','revenue','quantity']].dropna(),
                         x='age', y='revenue', covar='quantity')
print(result[['r','p-val','CI95%']].round(4))

# Compare raw vs partial
raw_r = df[['age','revenue']].corr().iloc[0,1]
print(f'Raw Pearson r(age, revenue) = {raw_r:.4f}')
print(f'Partial r(age, revenue | quantity) = {result.r.values[0]:.4f}')
```

■ ***If partial r differs substantially from raw r, quantity is a confounding variable in the age-revenue relationship.***

#101 Simple Linear Regression (OLS)

DEFINITION

Simple linear regression models the linear relationship between one predictor (X) and one response variable (Y). The OLS (Ordinary Least Squares) method finds coefficients that minimize the sum of squared residuals. The slope (β■) estimates the change in Y per unit change in X; the intercept (β■) is the predicted Y when X=0.

FORMULA

■ = β■ + β■X

β■ = Σ(x■–x■)(y■–■) / Σ(x■–x■)²

β■ = ■ – β■x■

WHEN TO USE

To model and quantify the linear relationship between one predictor and one continuous outcome.

PYTHON CODE

```
import statsmodels.formula.api as smf

model = smf.ols('revenue ~ age', data=df.dropna()).fit()
print(model.summary())
# Extract key metrics
print(f'R² = {model.rsquared:.4f}')
print(f'β■(age) = {model.params["age"]:.4f}')
print(f'p(age) = {model.pvalues["age"]:.4f}')
```

■ ***β■ = 2.5 means: for each additional year of age, predicted revenue increases by $2.50, holding all else equal.***

#102 Multiple Linear Regression

DEFINITION

Multiple linear regression extends simple regression to include two or more predictors. Each coefficient (β■) represents the change in Y per unit change in X■, holding all other predictors constant. The F-test assesses overall model significance; individual t-tests assess each predictor's significance.

FORMULA

■ = β■ + β■X■ + β■X■ + ... + β■X■

Minimize: Σ(y■ – ■■)²

WHEN TO USE

When multiple predictors collectively explain the outcome better than any single predictor alone.

PYTHON CODE

```
model = smf.ols('revenue ~ age + quantity + days_to_ship + C(gender) + C(category)',
                data=df.dropna()).fit()
print(model.summary())
# Coefficient plot
coef = model.params[1:].sort_values()
coef.plot(kind='barh', figsize=(8,6), title='Regression Coefficients')
plt.axvline(0, color='red', linestyle='--')
plt.show()
```

■ ***Positive coefficients increase predicted revenue; negative coefficients decrease it. Starred predictors (p < 0.05) are significant.***

#103 R² and Adjusted R²

DEFINITION

R² (coefficient of determination) measures the proportion of variance in Y explained by the model: R² = 1 – SSR/SST. It always increases when predictors are added. Adjusted R² penalizes for adding uninformative predictors: it increases only if the new predictor genuinely improves the model beyond what chance alone would achieve.

FORMULA

$R^2 = 1 - SSR/SST = 1 - \Sigma(y■ - ■■)^2 / \Sigma(y■ - ■)^2$

$Adj.R^2 = 1 - (1-R^2)(n-1)/(n-p-1)$

WHEN TO USE

To assess how well the regression model explains the outcome's variance.

PYTHON CODE

```
print(f'R² = {model.rsquared:.4f}')
print(f'Adjusted R² = {model.rsquared_adj:.4f}')
print(f'F-statistic = {model.fvalue:.4f}, p = {model.f_pvalue:.6f}')
# If adj.R² << R², many predictors are noise
```

■ ***R²=0.45 means the model explains 45% of revenue variance. If Adj.R² is much lower than R², consider removing weak predictors.***

#104 RMSE and MAE (Regression Error Metrics)

DEFINITION

RMSE (Root Mean Squared Error) is the square root of the mean squared prediction error. It penalizes large errors more than small ones. MAE (Mean Absolute Error) is the average absolute prediction error — more interpretable and less sensitive to outliers. Both are in the same units as the target variable.

FORMULA

RMSE = √(Σ(y■–■■)²/n)
MAE = (1/n) Σ|y■–■■|

WHEN TO USE

To measure the out-of-sample predictive accuracy of a regression model.

PYTHON CODE

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

y_true = df.dropna(subset=['revenue'])['revenue']
y_pred = model.predict(df.dropna(subset=['revenue']))
y_pred = y_pred[:len(y_true)]

mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
print(f'MAE = {mae:.2f}')
print(f'RMSE = {rmse:.2f}')
```

■ ***If RMSE >> MAE, there are a few large prediction errors dominating. Investigate those high-error cases for pattern insights.***

#105 Residual Analysis

DEFINITION

Residuals (e■ = y■ – ■■) should be: (1) normally distributed, (2) have constant variance (homoscedasticity), (3) be uncorrelated with predictors and each other (independence). Residual plots — residuals vs fitted, Q-Q of residuals, scale-location, and residuals vs leverage — collectively diagnose assumption violations.

FORMULA

e■ = y■ – ■■
H■ for good model: E(e■) = 0, Var(e■) = σ²

WHEN TO USE

After fitting any regression model, as a standard diagnostic step.

PYTHON CODE

```
residuals = model.resid
fitted = model.fittedvalues
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].scatter(fitted, residuals, alpha=0.4, s=15)
axes[0].axhline(0, color='red', ls='--')
axes[0].set_title('Residuals vs Fitted')
stats.probplot(residuals, dist='norm', plot=axes[1])
axes[1].set_title('Q-Q of Residuals')
pd.Series(residuals).hist(bins=40, ax=axes[2])
axes[2].set_title('Residual Distribution')
plt.tight_layout(); plt.show()
```

■ ***Fan-shaped residuals vs fitted → heteroscedasticity. Curved pattern → non-linearity. Fat tails in Q-Q → non-normal residuals.***

#106 Variance Inflation Factor (VIF)

DEFINITION

VIF measures multicollinearity: how much the variance of a regression coefficient is inflated due to correlation with other predictors. VIF = 1/(1–R²■), where R²■ is the R² from regressing predictor j on all other predictors. Rule of thumb: VIF > 5 is a concern; VIF > 10 indicates severe multicollinearity.

FORMULA

VIF■ = 1 / (1 – R²■)

WHEN TO USE

After fitting a multiple regression model to check for multicollinearity among predictors.

PYTHON CODE

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

X = df[['age','quantity','unit_price','days_to_ship']].dropna()
X_const = sm.add_constant(X)
vif_df = pd.DataFrame({'Feature': X.columns,
                       'VIF': [variance_inflation_factor(X_const.values, i+1)
                               for i in range(len(X.columns))]})
print(vif_df.sort_values('VIF', ascending=False))
```

■ ***VIF > 10: drop or combine the correlated predictors. High VIF makes individual coefficient estimates unreliable and unstable.***

#107 Breusch-Pagan Test (Heteroscedasticity)

DEFINITION

The Breusch-Pagan test formally tests for heteroscedasticity: whether residual variance is constant (homoscedastic) or varies with fitted values. H■: homoscedasticity (constant variance). A significant result (p < 0.05) indicates heteroscedasticity, which makes OLS standard errors unreliable (use robust standard errors instead).

FORMULA

BP statistic ~ $\chi^2(p)$ where p = number of predictors
Based on R^2 from regressing squared residuals on predictors

WHEN TO USE

As a formal test following residual vs fitted plot inspection.

PYTHON CODE

```
from statsmodels.stats.diagnostic import het_breuschpagan
import statsmodels.api as sm

X = sm.add_constant(df[['age','quantity']].dropna())
y = df.loc[df[['age','quantity']].dropna().index, 'revenue']
model_bp = sm.OLS(y, X).fit()
bp_stat, bp_p, f_stat, f_p = het_breuschpagan(model_bp.resid, model_bp.model.exog)
print(f'Breusch-Pagan: LM={bp_stat:.4f}, p={bp_p:.4f}')
if bp_p < 0.05: print('Heteroscedasticity detected → use robust SE')
```

■ ***If heteroscedasticity is detected, use HC3 robust standard errors: sm.OLS(...).fit(cov_type='HC3').***

#108 Durbin-Watson Test (Autocorrelation)

DEFINITION

The Durbin-Watson (DW) statistic tests for first-order autocorrelation in regression residuals. Values near 2 indicate no autocorrelation. DW < 1.5 suggests positive autocorrelation (consecutive residuals tend to be similar); DW > 2.5 suggests negative autocorrelation. Autocorrelated residuals violate the OLS independence assumption, inflating significance of regression coefficients.

FORMULA

DW = Σ■■■■(e■–e■■■)² / Σ■■■■e■²

DW ∈ [0, 4] | DW ≈ 2 → no autocorrelation

WHEN TO USE

For regression on time-series or ordered data, where residual independence is especially critical.

PYTHON CODE

```
from statsmodels.stats.stattools import durbin_watson

dw = durbin_watson(model.resid)
print(f'Durbin-Watson statistic: {dw:.4f}')
if dw < 1.5:
    print('Positive autocorrelation detected')
elif dw > 2.5:
    print('Negative autocorrelation detected')
else:
    print('No significant autocorrelation')
```

■ ***DW < 1.5 in a time-series regression indicates the model is missing a temporal pattern — consider adding lag features or using ARIMA.***

#109 Chi-Square Test of Independence

DEFINITION

The Chi-square test of independence tests whether two categorical variables are statistically independent (i.e., whether their joint distribution equals the product of their marginal distributions). H■: the two variables are independent. Expected cell frequencies under H■: E■■ = (row_total × col_total) / grand_total. A significant result means the variables are associated.

FORMULA

χ^2 = Σ (O■■ – E■■)² / E■■

df = (r–1)(c–1) | H■: variables are independent

WHEN TO USE

To test association between two categorical variables when cell counts are sufficient (expected count ≥ 5 per cell).

PYTHON CODE

```
from scipy.stats import chi2_contingency

ct = pd.crosstab(df['gender'], df['category'])
chi2, p, dof, expected = chi2_contingency(ct)
print(f'Chi-square: χ²={chi2:.4f}, p={p:.4f}, df={dof}')
if p < 0.05:
    print('Gender and Category are NOT independent (significant association)')
# Check expected frequencies
print('Min expected frequency:', expected.min().round(2))
```

■ ***Rule: if any expected cell < 5, use Fisher's Exact Test instead. |χ^2| alone is not an effect size — compute Cramér's V.***

#110 Chi-Square Goodness of Fit

DEFINITION

The Chi-square goodness-of-fit test determines whether an observed frequency distribution differs significantly from a theoretical (expected) distribution. For example: are returns distributed equally across the week, or do some days have more returns? H■: the observed frequencies match the expected frequencies.

FORMULA

$\chi^2 = \Sigma$ (O■ – E■)² / E■

df = k – 1 where k = number of categories

WHEN TO USE

To test if a categorical variable follows a hypothesized distribution (e.g., uniform, or a specified ratio).

PYTHON CODE

```
from scipy.stats import chisquare

# Test if orders are uniformly distributed across months
obs = df['month'].value_counts().sort_index().values
expected = np.full(12, len(df) / 12)  # uniform expectation
chi2, p = chisquare(f_obs=obs, f_exp=expected)
print(f'Goodness of Fit: χ²={chi2:.4f}, p={p:.4f}')
if p < 0.05: print('Orders are NOT uniformly distributed across months')
```

Significant result = the observed distribution differs from the expected one. Identify which months have higher/lower than expected counts.

#111 Cramér's V (Effect Size for Chi-Square)

DEFINITION

Cramér's V is an effect size measure for the chi-square test of independence. It measures the strength of association between two categorical variables, regardless of table size. V ranges from 0 (no association) to 1 (perfect association). Guidelines: 0.1=small, 0.3=medium, 0.5=large.

FORMULA

$V = \sqrt{(\chi^2/n \times \min(r-1, c-1))}$

n = total observations, r = rows, c = columns

WHEN TO USE

Always report alongside chi-square test results to convey effect magnitude.

PYTHON CODE

```
def cramers_v(chi2, n, r, c):
    return np.sqrt(chi2 / (n * (min(r, c) - 1)))

ct = pd.crosstab(df['gender'], df['category'])
chi2, p, dof, _ = chi2_contingency(ct)
V = cramers_v(chi2, len(df), ct.shape[0], ct.shape[1])
print(f"Cramér's V = {V:.4f}")
print(f'Interpretation: {"small" if V<0.1 else "medium" if V<0.3 else "large"} effect')
```

V=0.15 means a small-to-medium association between gender and category. Chi-square significance alone doesn't tell you this.

#112 Fisher's Exact Test

DEFINITION

Fisher's exact test is used for 2×2 contingency tables when expected cell frequencies are small (< 5). Unlike chi-square which uses a distributional approximation, Fisher's test computes the exact probability of observing the table (and more extreme tables). It is the gold standard for small samples in 2×2 comparisons.

FORMULA

P = (r■!r■!c■!c■!) / (n! × Πn■■!)

WHEN TO USE

For 2×2 tables with small expected frequencies (any cell expected count < 5).

PYTHON CODE

```
from scipy.stats import fisher_exact

# 2x2 contingency: return rate by gender
ct = pd.crosstab(df['gender'], df['is_returned'])
print(ct)
oddsratio, p = fisher_exact(ct)
print(f"Fisher's Exact: OR={oddsratio:.4f}, p={p:.4f}")
```

Odds ratio > 1 means the first row (e.g., Female) has higher odds of the outcome (e.g., return). OR=2 means twice the odds.

#113 Phi Coefficient

DEFINITION

The Phi (ϕ) coefficient measures the association between two binary variables. It is equivalent to the Pearson correlation applied to two dummy variables and to Cramér's V for a 2×2 table. Range: –1 to +1. Guidelines: 0.1=small, 0.3=medium, 0.5=large.

FORMULA

ϕ = (ad – bc) / √((a+b)(c+d)(a+c)(b+d))

a,b,c,d = cells of 2×2 table

WHEN TO USE

When both variables are binary (yes/no, 0/1) — a special case of Cramér's V.

PYTHON CODE

```
from scipy.stats import chi2_contingency
from sklearn.metrics import matthews_corrcoef

# Phi is MCC for binary outcomes
df['premium_customer'] = (df['revenue'] > df['revenue'].quantile(0.75)).astype(int)
phi = matthews_corrcoef(df['is_returned'].dropna(), df['premium_customer'].dropna())
print(f'Phi coefficient = {phi:.4f}')
```

Phi is the MCC (Matthews Correlation Coefficient) for binary outcomes — a balanced measure even for imbalanced datasets.

#114 Z-Test for Proportions

DEFINITION

The two-proportion z-test tests whether two groups have significantly different proportions for a binary outcome. For example: 'Is the return rate significantly different between male and female customers?' The z-statistic uses the pooled proportion under H■.

FORMULA

z = (p■■ – p■■) / √(p■(1–p■)(1/n■ + 1/n■))
p■ = pooled proportion

WHEN TO USE

To compare two proportions (e.g., conversion rates, return rates) between two independent groups.

PYTHON CODE

```
from statsmodels.stats.proportion import proportions_ztest

returns_m = (df[df.gender=='Male']['order_status'] == 'Returned').sum()
n_m = df[df.gender=='Male']['order_status'].notna().sum()
returns_f = (df[df.gender=='Female']['order_status'] == 'Returned').sum()
n_f = df[df.gender=='Female']['order_status'].notna().sum()
stat, p = proportions_ztest([returns_m, returns_f], [n_m, n_f])
print(f'Z={stat:.4f}, p={p:.4f}')
print(f'Male return rate: {returns_m/n_m:.3f}')
print(f'Female return rate: {returns_f/n_f:.3f}')
```

■ ***If p < 0.05, the return rates are significantly different. Report the actual rate difference alongside the p-value.***

#115 McNemar's Test

DEFINITION

McNemar's test compares two related proportions — specifically, before/after or matched-pair binary outcomes on the same subjects. It focuses on the discordant pairs (where outcomes differ between conditions). Used in pre/post studies, diagnostic test comparisons, and matched case-control designs.

FORMULA

$\chi^2 = (b - c)^2 / (b + c)$
b = cases: outcome1=Yes, outcome2=No
c = cases: outcome1=No, outcome2=Yes

WHEN TO USE

When comparing binary outcomes from two related/paired measurements (not independent groups).

PYTHON CODE

```
from statsmodels.stats.contingency_tables import mcnemar

# Simulate pre/post table
table = [[50, 30],  # [both_yes, before_yes_after_no]
         [20, 100]] # [before_no_after_yes, both_no]
result = mcnemar(table, exact=False)
print(f"McNemar: χ²={result.statistic:.4f}, p={result.pvalue:.4f}")
```

■ ***A significant McNemar test means the proportion of 'yes' outcomes changed significantly between condition 1 and condition 2.***

#116 Summary Statistics Table

DEFINITION

A final structured table aggregating key statistical metrics for all relevant variables: count, mean, median, std, skewness, kurtosis, min, max, and missing percentage. This single table provides a complete numerical overview and serves as the anchor table in any statistical analysis report.

FORMULA

N/A

WHEN TO USE

As a presentation-ready summary table at the end of the statistical analysis section.

PYTHON CODE

```
def full_summary(df):
    num_df = df.select_dtypes(include=np.number)
    summary = num_df.describe().T
    summary['skewness'] = num_df.skew()
    summary['kurtosis'] = num_df.kurt()
    summary['missing_pct'] = (df.isnull().sum() / len(df) * 100).round(2)
    return summary.round(3)

print(full_summary(df).to_string())
```

Use this table as the primary reference table in statistical reports. Share it with stakeholders before presenting visualizations.

Chapter 4

Segmentation & Clustering

RFM, K-Means, PCA, DBSCAN

33 Concept

#117 RFM Analysis Introduction

DEFINITION

RFM (Recency, Frequency, Monetary) analysis is a customer segmentation framework that quantifies customer value along three behavioral dimensions. It is grounded in the direct marketing principle that customers who bought recently, buy often, and spend the most are most likely to respond to future campaigns. Each customer receives scores on all three dimensions, which are then combined to create actionable segments.

FORMULA

R = days since last purchase (lower = better)
F = total number of transactions
M = total spend

WHEN TO USE

For customer segmentation in retail, e-commerce, and subscription businesses to prioritize marketing spend.

PYTHON CODE

```
import pandas as pd
reference_date = df['order_date'].max() + pd.Timedelta(days=1)
rfm = df.groupby('customer_id').agg(
    Recency=('order_date', lambda x: (reference_date - x.max()).days),
    Frequency=('order_id', 'count'),
    Monetary=('revenue', 'sum')
).reset_index()
print(rfm.head())
print(rfm.describe().round(2))
```

A customer with R=5, F=12, M=1500 is a high-value, engaged recent buyer — prime target for loyalty programs.

#118 Recency Calculation

DEFINITION

Recency measures how many days have elapsed since a customer's last purchase, calculated from a reference date (usually the day after the last transaction in the dataset). Lower recency = more recent purchase = more engaged customer. Recency is often the strongest predictor of future purchase probability.

FORMULA

Recency = reference_date – max(order_date per customer)

WHEN TO USE

As part of RFM analysis, or any churn prediction model.

PYTHON CODE

```
reference_date = df['order_date'].max() + pd.Timedelta(days=1)
recency = df.groupby('customer_id')['order_date'].max()
recency = (reference_date - recency).dt.days.rename('Recency')
print(f'Recency range: {recency.min()}–{recency.max()} days')
print(f'Median recency: {recency.median():.0f} days')
```

Customers with recency > 365 days are likely churned. Customers with recency < 30 days are highly engaged.

#119 Frequency Calculation

DEFINITION

Frequency counts the number of distinct transactions (or visits) per customer within the analysis period. High frequency indicates loyalty and habitual purchasing. Combined with recency, frequency helps distinguish between loyal customers who haven't bought recently (high F, high R) from new customers who bought once recently (low F, low R).

FORMULA

Frequency = count(order_id per customer)

WHEN TO USE

As part of RFM analysis and customer lifetime value estimation.

PYTHON CODE

```
frequency = df.groupby('customer_id')['order_id'].count().rename('Frequency')
print(f'Max frequency: {frequency.max()}')
print(f'% one-time buyers: {(frequency == 1).mean() * 100:.1f}%')
frequency.hist(bins=30, edgecolor='white')
plt.title('Distribution of Purchase Frequency')
plt.xlabel('Number of Orders'); plt.show()
```

In many retail datasets, 40–60% of customers buy only once. These single-purchase customers need win-back campaigns.

#120 Monetary Calculation

DEFINITION

Monetary value is the total revenue generated by a customer over the analysis period. It is the primary indicator of customer economic value. Combined with frequency, it yields average order value (Monetary/Frequency), which helps distinguish high-value infrequent buyers from lower-value frequent buyers.

FORMULA

Monetary = sum(revenue per customer)

WHEN TO USE

For identifying high-value customers (VIPs) and calculating Customer Lifetime Value (CLV).

PYTHON CODE

```
monetary = df.groupby('customer_id')['revenue'].sum().rename('Monetary')
print(f'Top 10% customers account for: '
      f'{monetary.quantile(0.9)/monetary.sum()*100:.1f}% of revenue')
# Pareto check (80/20 rule)
sorted_m = monetary.sort_values(ascending=False)
cum_pct = sorted_m.cumsum() / sorted_m.sum()
top20pct_customers = int(0.2 * len(monetary))
print(f'Top 20% customers = {cum_pct.iloc[top20pct_customers]*100:.1f}% of revenue')
```

If the top 20% of customers generate >60% of revenue, focus retention efforts on this segment.

#121 RFM Quintile Scoring

DEFINITION

Quintile scoring assigns each customer an RFM score from 1–5 by dividing each RFM dimension into five equal-frequency bins (quintiles using pd.qcut). For Recency, score 5 is assigned to the most recent customers (low raw recency value). For Frequency and Monetary, score 5 is assigned to the highest values. The combined RFM score = R_score + F_score + M_score (range: 3–15).

FORMULA

R_score = quintile rank (5 = most recent)
F_score = quintile rank (5 = most frequent)
M_score = quintile rank (5 = highest monetary)

WHEN TO USE

To normalize and combine three different-scale RFM dimensions into a single comparable score.

PYTHON CODE

```
rfm['R_score'] = pd.qcut(rfm['Recency'], q=5, labels=[5,4,3,2,1])
rfm['F_score'] = pd.qcut(rfm['Frequency'].rank(method='first'), q=5, labels=[1,2,3,4,5])
rfm['M_score'] = pd.qcut(rfm['Monetary'].rank(method='first'), q=5, labels=[1,2,3,4,5])
rfm[['R_score','F_score','M_score']] = rfm[['R_score','F_score','M_score']].astype(int)
rfm['RFM_Score'] = rfm['R_score'] + rfm['F_score'] + rfm['M_score']
print(rfm.head())
```

RFM Score 13–15 = Champions. Score 3–5 = Lost customers. Score 8–12 = At-risk or Potential Loyalists.

#122 RFM Segment Labels

DEFINITION

After scoring, customers are assigned to named segments based on their RFM score profile. Common segments: Champions (555), Loyal Customers (high F, high M), At Risk (past buyers slipping away), Lost (very low R), New Customers (high R, low F). Segment labels make results actionable for marketing teams who may not interpret numeric scores.

FORMULA

Segment = f(R_score, F_score, M_score) — business-defined rules

WHEN TO USE

After RFM scoring, to translate scores into business-actionable customer groups.

PYTHON CODE

```
def rfm_label(row):
    if row['RFM_Score'] >= 13:
        return 'Champions'
    elif row['RFM_Score'] >= 10:
        return 'Loyal Customers'
    elif row['R_score'] >= 4 and row['F_score'] < 3:
        return 'New Customers'
    elif row['R_score'] <= 2 and row['F_score'] >= 3:
        return 'At Risk'
    elif row['RFM_Score'] <= 5:
        return 'Lost'
    else:
        return 'Potential Loyalists'
rfm['Segment'] = rfm.apply(rfm_label, axis=1)
print(rfm['Segment'].value_counts())
```

Share segment sizes with the marketing team. Champions warrant VIP treatment; At Risk need win-back emails.

#123 RFM Heatmap Visualization

DEFINITION

A heatmap of mean Monetary value across R_score (rows) and F_score (columns) reveals the interaction between recency and frequency in driving revenue. Another useful visualization is a scatter plot of Recency vs Monetary, colored by Frequency, to show all three dimensions simultaneously.

FORMULA

N/A

WHEN TO USE

To present RFM segment patterns to stakeholders in an intuitive visual format.

PYTHON CODE

```
pivot_rfm = rfm.pivot_table(values='Monetary', index='R_score',
                            columns='F_score', aggfunc='mean')
plt.figure(figsize=(8, 6))
sns.heatmap(pivot_rfm, annot=True, fmt='.0f',
            cmap='YlOrRd', linewidths=0.5)
plt.title('Mean Monetary Value: R_score × F_score')
plt.ylabel('Recency Score (5=Most Recent)')
plt.xlabel('Frequency Score (5=Most Frequent)')
plt.show()
```

Top-right cell (R=5, F=5) should have the highest monetary value — these are your Champions. Verify this in your data.

#124 Feature Engineering for Clustering

DEFINITION

Before applying clustering algorithms, raw features must be engineered to be informative, clean, and appropriately scaled. This involves: selecting relevant features, creating ratio features (e.g., avg_order_value = Monetary/Frequency), removing correlated features to avoid double-counting, and log-transforming skewed features to reduce the influence of extreme values.

FORMULA

N/A

WHEN TO USE

As the preprocessing step before any clustering algorithm.

PYTHON CODE

```
# Log transform skewed RFM features
rfm['log_Recency']   = np.log1p(rfm['Recency'])
rfm['log_Frequency'] = np.log1p(rfm['Frequency'])
rfm['log_Monetary']  = np.log1p(rfm['Monetary'])
rfm['avg_order_val'] = rfm['Monetary'] / rfm['Frequency']
features = ['log_Recency', 'log_Frequency', 'log_Monetary']
print(rfm[features].skew().round(3))
```

After log transform, |skewness| should drop below 1. Features with skew > 2 after log transform may need further treatment.

#125 Feature Scaling (StandardScaler, MinMaxScaler)

DEFINITION

Clustering algorithms based on distance metrics (K-Means, DBSCAN) are highly sensitive to feature scale. A feature with range 0–1000 will dominate a feature with range 0–1. StandardScaler standardizes features to zero mean and unit variance (Z-scores). MinMaxScaler scales to [0, 1]. Always scale before clustering.

FORMULA

StandardScaler: $z = (x - \mu) / \sigma$

MinMaxScaler: $x\_scaled = (x - min) / (max - min)$

WHEN TO USE

Before any distance-based clustering or PCA.

PYTHON CODE

```
from sklearn.preprocessing import StandardScaler

features = ['log_Recency', 'log_Frequency', 'log_Monetary']
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm[features])
rfm_scaled = pd.DataFrame(rfm_scaled, columns=features)
print(f'Mean after scaling: {rfm_scaled.mean().round(6).values}')
print(f'Std after scaling: {rfm_scaled.std().round(3).values}')
```

After StandardScaler, mean ≈ 0 and std ≈ 1 for all features. Check this to confirm scaling was applied correctly.

#126 Encoding Categorical Variables

DEFINITION

K-Means and hierarchical clustering require numeric input. Nominal categorical variables must be encoded: one-hot encoding (pd.get_dummies) creates binary columns for each category. Ordinal categories can be label-encoded with meaningful integer order. Be careful: one-hot encoding many categories bloats the feature space.

FORMULA

One-hot: category → binary vector of length k

Label encode: category → integer 0..k-1

WHEN TO USE

When including categorical features in distance-based clustering.

PYTHON CODE

```
from sklearn.preprocessing import LabelEncoder

# One-hot for nominal categories
encoded = pd.get_dummies(df[['gender','category','payment_method']], drop_first=True)
print(encoded.shape)

# Label encode for ordinal
le = LabelEncoder()
df['payment_encoded'] = le.fit_transform(df['payment_method'].fillna('Unknown'))
```

After one-hot encoding, scale the binary columns along with numeric features. High-cardinality one-hot encoding can dominate the distance computation.

#127 K-Means Clustering Algorithm

DEFINITION

K-Means partitions n observations into k clusters by iteratively: (1) Assigning each point to its nearest centroid (by Euclidean distance), and (2) Recomputing centroids as the mean of all points in each cluster. Convergence occurs when assignments no longer change. K-Means assumes spherical, roughly equal-size clusters and is sensitive to initialization and outliers.

FORMULA

Minimize: Σ■ Σ■∈C■ ||x■ – μ■||²
μ■ = centroid of cluster k

WHEN TO USE

For partitioning continuous, scaled data into k groups when k is known or can be estimated.

PYTHON CODE

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
rfm['Cluster'] = kmeans.fit_predict(rfm_scaled)
print('Cluster sizes:')
print(rfm['Cluster'].value_counts().sort_index())
print(f'Inertia: {kmeans.inertia_:.2f}')
```

Cluster sizes should be reasonably balanced. Very small clusters (< 5% of data) may indicate noise or outliers that K-Means forced into a cluster.

#128 Choosing K — Elbow Method (Inertia)

DEFINITION

The Elbow Method plots within-cluster sum of squares (inertia) vs k. Inertia measures total distance of points from their cluster centroids — it always decreases as k increases. The 'elbow' point where inertia begins decreasing less rapidly suggests the optimal k, after which adding more clusters provides diminishing returns.

FORMULA

Inertia = Σ■ Σ■∈C■ ||x■ – μ■||²
Choose k at the 'elbow' of the inertia curve

WHEN TO USE

As the first method to estimate optimal k for K-Means.

PYTHON CODE

```
inertias = []
K_range = range(2, 11)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(rfm_scaled)
    inertias.append(km.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(K_range, inertias, 'o-', color='steelblue')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia (WCSS)')
plt.title('Elbow Method for Optimal k')
plt.xticks(K_range); plt.grid(alpha=0.3); plt.show()
```

The elbow is often subjective. Use the Silhouette Score in addition to confirm the optimal k.

#129 Choosing K — Silhouette Score

DEFINITION

The Silhouette Score measures how similar each point is to its own cluster compared to other clusters. It ranges from –1 to +1: +1 = well-clustered; 0 = on border; –1 = assigned to wrong cluster. The average silhouette score across all samples provides a single quality metric. The optimal k is typically where this score is maximized.

FORMULA

s(i) = (b(i) – a(i)) / max(a(i), b(i))
a(i) = mean intra-cluster distance, b(i) = nearest inter-cluster distance

WHEN TO USE

To objectively compare clustering quality across different values of k.

PYTHON CODE

```
from sklearn.metrics import silhouette_score

sil_scores = []
K_range = range(2, 11)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(rfm_scaled)
    sil_scores.append(silhouette_score(rfm_scaled, labels))

plt.figure(figsize=(8, 5))
plt.plot(K_range, sil_scores, 'o-', color='emerald')
plt.xlabel('k'); plt.ylabel('Silhouette Score')
plt.title('Silhouette Score vs k')
best_k = K_range[np.argmax(sil_scores)]
plt.axvline(best_k, color='red', ls='--', label=f'Best k={best_k}')
plt.legend(); plt.show()
```

Silhouette > 0.5 indicates good cluster separation. < 0.25 suggests clusters are poorly defined.

#130 Silhouette Plot

DEFINITION

A silhouette plot shows the silhouette coefficient for every individual data point, organized by cluster. Each cluster's bar width represents cluster size, and bar length represents silhouette value. Clusters with many negative-silhouette points or very thin bars compared to others suggest poor clustering quality.

FORMULA

Same as silhouette score but plotted per-point

WHEN TO USE

To diagnose the quality of individual clusters, not just overall clustering.

PYTHON CODE

```
from sklearn.metrics import silhouette_samples

km = KMeans(n_clusters=4, random_state=42, n_init=10)
labels = km.fit_predict(rfm_scaled)
sil_vals = silhouette_samples(rfm_scaled, labels)
avg_sil = sil_vals.mean()
y_lower = 10
fig, ax = plt.subplots(figsize=(8, 6))
for i in range(4):
    cluster_sil = np.sort(sil_vals[labels == i])
    y_upper = y_lower + len(cluster_sil)
    ax.fill_betweenx(np.arange(y_lower, y_upper), 0, cluster_sil)
    y_lower = y_upper + 10
ax.axvline(avg_sil, color='red', ls='--', label=f'Avg: {avg_sil:.3f}')
ax.set_title('Silhouette Plot (k=4)'); ax.legend(); plt.show()
```

Clusters where many points have silhouette < average line should be re-examined. They may need to be merged with adjacent clusters.

#131 K-Means Cluster Profiling

DEFINITION

After assigning clusters, profile each cluster by computing the mean (or median) of key features per cluster. This answers: 'What makes Cluster 2 different from Cluster 3?' Sorted profiling tables and spider/radar charts are the most effective tools for communicating cluster characteristics to business stakeholders.

FORMULA

N/A

WHEN TO USE

After running K-Means (or any clustering), to interpret and name the resulting clusters.

PYTHON CODE

```
profile = rfm.groupby('Cluster')[['Recency','Frequency','Monetary']]              .agg(['mean','median','count
print(profile.round(2))

# Radar/spider chart for cluster profiles
from matplotlib.patches import FancyArrowPatch
fig, ax = plt.subplots(figsize=(10, 5))
rfm.groupby('Cluster')[['Recency','Frequency','Monetary']]   .mean().T.plot(kind='bar', ax=ax)
ax.set_title('Mean RFM Values per Cluster')
plt.tight_layout(); plt.show()
```

Name clusters after their dominant characteristic: Cluster with high F + high M = 'Champions'; low R (high days) + low F = 'Lost'.

#132 Principal Component Analysis (PCA)

DEFINITION

PCA is a dimensionality reduction technique that projects high-dimensional data onto orthogonal axes (principal components) that capture maximum variance. PC1 explains the most variance, PC2 the second most, and so on. PCA is used for: (1) visualization of high-dimensional clusters in 2D/3D, (2) feature compression before clustering, and (3) removing multicollinearity.

FORMULA

Cov(X) = V · Λ · V■ (eigendecomposition)
PC■ = X■ projected onto eigenvector i

WHEN TO USE

When the feature space has > 3 dimensions and you need visualization or dimensionality reduction.

PYTHON CODE

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2, random_state=42)
pca_coords = pca.fit_transform(rfm_scaled)
rfm['PC1'] = pca_coords[:, 0]
rfm['PC2'] = pca_coords[:, 1]
print(f'Explained variance: PC1={pca.explained_variance_ratio_[0]:.2%}, '
      f'PC2={pca.explained_variance_ratio_[1]:.2%}')
plt.figure(figsize=(8, 6))
sns.scatterplot(data=rfm, x='PC1', y='PC2', hue='Cluster',
                palette='Set2', alpha=0.7, s=40)
plt.title('K-Means Clusters in PCA Space')
plt.show()
```

■ ***Well-separated color clusters in PCA space confirm good cluster quality. Overlapping clusters signal ambiguity.***

#133 Explained Variance Ratio

DEFINITION

The explained variance ratio for each principal component indicates what fraction of total data variance it captures. The cumulative explained variance plot shows how many components are needed to explain, say, 90% of the variance. This guides the choice of how many components to retain in dimensionality reduction.

FORMULA

EVR■ = λ■ / Σλ■

Cumulative EVR = Σ■■■■ EVR■

WHEN TO USE

To decide how many principal components to retain for further analysis or modeling.

PYTHON CODE

```
pca_full = PCA(random_state=42)
pca_full.fit(rfm_scaled)
cum_var = np.cumsum(pca_full.explained_variance_ratio_)
plt.figure(figsize=(8, 5))
plt.bar(range(1, len(cum_var)+1),
        pca_full.explained_variance_ratio_, label='Individual', alpha=0.6)
plt.plot(range(1, len(cum_var)+1), cum_var, 'ro-', label='Cumulative')
plt.axhline(0.9, ls='--', color='gray', label='90% threshold')
plt.xlabel('Principal Component'); plt.ylabel('Variance Explained')
plt.legend(); plt.title('Scree Plot + Cumulative Variance')
plt.show()
```

■ ***Choose the smallest k components that achieve ≥90% cumulative explained variance. For 3 RFM features, often 2 PCs suffice.***

#134 Scree Plot

DEFINITION

A scree plot graphs the eigenvalues (or explained variance) of each principal component in descending order. The 'elbow' in the plot (where the decline levels off) marks the natural cutoff for component retention — similar conceptually to the Elbow Method for K-Means. Components to the right of the elbow contribute little new information.

FORMULA

Eigenvalue λ■ = variance explained by PC■

WHEN TO USE

To visually determine how many principal components to retain.

PYTHON CODE

```
eigenvalues = pca_full.explained_variance_
plt.figure(figsize=(8, 5))
plt.plot(range(1, len(eigenvalues)+1), eigenvalues, 'o-', color='steelblue')
plt.axhline(1, ls='--', color='red', label='Kaiser criterion (λ=1)')
plt.xlabel('Component Number')
plt.ylabel('Eigenvalue')
plt.title('Scree Plot')
plt.legend(); plt.grid(alpha=0.3); plt.show()
```

■ ***Kaiser criterion: retain components with eigenvalue > 1. This is a common rule of thumb for standardized data.***

#135 PCA Biplot

DEFINITION

A PCA biplot overlays both the sample scores (plotted as points in PC space) and the variable loadings (plotted as arrows) on the same chart. Arrow direction shows which features contribute most to each PC; arrow length indicates the magnitude of contribution. Points in the direction of an arrow tend to have high values for that feature.

FORMULA

Loading = correlation between original variable and PC

Arrow length ∝ contribution to variance

WHEN TO USE

To simultaneously visualize both the cluster structure and the feature importance in PCA space.

PYTHON CODE

```
pca_2d = PCA(n_components=2, random_state=42)
scores = pca_2d.fit_transform(rfm_scaled)
loadings = pca_2d.components_.T
features_list = ['log_Recency','log_Frequency','log_Monetary']
fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(scores[:,0], scores[:,1], alpha=0.3, s=15)
for i, feat in enumerate(features_list):
    ax.arrow(0, 0, loadings[i,0]*3, loadings[i,1]*3,
             head_width=0.1, color='red')
    ax.text(loadings[i,0]*3.2, loadings[i,1]*3.2, feat, fontsize=10)
ax.set_title('PCA Biplot'); plt.show()
```

Features whose arrows point in the same direction are positively correlated. Orthogonal arrows indicate uncorrelated features.

#136 Hierarchical Clustering

DEFINITION

Hierarchical clustering builds a tree (dendrogram) of clusters by iteratively merging (agglomerative) or splitting (divisive) clusters. Unlike K-Means, it does not require specifying k in advance. The dendrogram reveals the full hierarchy of cluster relationships and allows cutting at any desired level to obtain any number of clusters.

FORMULA

Distance(Cluster A, Cluster B) = linkage function(pairwise distances)
Common linkages: single, complete, average, Ward's

WHEN TO USE

When you want to explore the cluster hierarchy, or when k is unknown and you want visual guidance.

PYTHON CODE

```
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.preprocessing import StandardScaler

sample = rfm_scaled.sample(200, random_state=42)
Z = linkage(sample, method='ward', metric='euclidean')
plt.figure(figsize=(14, 6))
dendrogram(Z, truncate_mode='lastp', p=30,
           leaf_rotation=90, leaf_font_size=10)
plt.title("Hierarchical Clustering Dendrogram (Ward's)")
plt.xlabel('Cluster or Point'); plt.ylabel('Distance')
plt.show()
```

The height of each merge in the dendrogram indicates the dissimilarity at which clusters were joined. Large jumps suggest natural cluster boundaries.

#137 Linkage Methods

DEFINITION

The linkage method determines how distance between clusters is computed when merging. Single linkage: minimum pairwise distance (chaining-prone). Complete linkage: maximum pairwise distance (compact clusters). Average linkage: mean of all pairwise distances. Ward's method: minimizes within-cluster variance increase — produces compact, equal-size clusters and is the most commonly recommended method.

FORMULA

Ward: $\Delta\text{Error} = (n_A n_B/(n_A+n_B)) \times ||\mu_A - \mu_B||^2$

Single: $\min(d(a,b))$ for $a \in A$, $b \in B$

WHEN TO USE

Ward's linkage is the default choice. Use average for elongated clusters, complete for well-separated compact clusters.

PYTHON CODE

```
from scipy.cluster.hierarchy import linkage
import matplotlib.pyplot as plt
fig, axes = plt.subplots(1, 4, figsize=(20, 5))
methods = ['single', 'complete', 'average', 'ward']
for ax, method in zip(axes, methods):
    Z = linkage(sample, method=method)
    from scipy.cluster.hierarchy import dendrogram
    dendrogram(Z, ax=ax, truncate_mode='lastp', p=20,
               no_labels=True)
    ax.set_title(f'{method.title()} Linkage')
plt.tight_layout(); plt.show()
```

Ward's dendrogram typically shows the clearest elbow-like structure for cutting into clusters.

#138 Dendrogram

DEFINITION

A dendrogram is a tree diagram that visually represents the hierarchical merging sequence. The y-axis shows the distance (dissimilarity) at which each merge occurred. The x-axis represents individual observations or sub-clusters. To choose the number of clusters, draw a horizontal line at a large vertical jump — the number of vertical lines it crosses is the number of clusters.

FORMULA

N/A (visual representation of linkage hierarchy)

WHEN TO USE

After hierarchical clustering, to determine the optimal number of clusters by visual inspection.

PYTHON CODE

```
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

Z = linkage(rfm_scaled.sample(300, random_state=42), method='ward')
plt.figure(figsize=(14, 7))
d = dendrogram(Z, truncate_mode='level', p=5, color_threshold=10)
plt.title("Ward's Dendrogram (300 customers)")
plt.xlabel('Sample or [Cluster Size]')
plt.ylabel('Euclidean Distance (Ward)')
plt.axhline(10, color='red', ls='--', label='Cut threshold')
plt.legend(); plt.show()
```

The optimal cut is made below the longest vertical line that is not crossed by any horizontal line — this represents the most natural cluster structure.

#139 Cutting the Dendrogram (fcluster)

DEFINITION

scipy.cluster.hierarchy.fcluster() cuts the dendrogram to obtain flat cluster assignments. Two cutting criteria: (1) distance threshold (t=distance, criterion='distance') — cuts where linkage exceeds t; (2) number of clusters (t=k, criterion='maxclust') — cuts to produce exactly k clusters. The output is a 1D array of cluster labels.

FORMULA

N/A — algorithmic cutting of dendrogram at specified level

WHEN TO USE

After visual inspection of the dendrogram to obtain actual cluster assignments.

PYTHON CODE

```
from scipy.cluster.hierarchy import fcluster

Z = linkage(rfm_scaled, method='ward')
# Cut to 4 clusters
labels_hier = fcluster(Z, t=4, criterion='maxclust')
rfm['Hier_Cluster'] = labels_hier
print('Hierarchical cluster sizes:')
print(pd.Series(labels_hier).value_counts().sort_index())
```

Compare hierarchical cluster assignments with K-Means using Adjusted Rand Index (ARI). High ARI means both methods agree.

#140 Agglomerative vs Divisive Clustering

DEFINITION

Agglomerative clustering starts with each point as its own cluster and successively merges the two closest clusters until all points form one cluster (bottom-up). Divisive clustering starts with all points in one cluster and recursively splits the least cohesive cluster (top-down). Agglomerative is far more common and computationally efficient for most datasets.

FORMULA

Agglomerative: O(n² log n) time complexity
Divisive: O(2■) exact; approximations used in practice

WHEN TO USE

Agglomerative is the standard choice. Divisive is rarely needed except when top-level splits are more important than fine-grained merges.

PYTHON CODE

```
from sklearn.cluster import AgglomerativeClustering

agg = AgglomerativeClustering(n_clusters=4, linkage='ward')
rfm['Agg_Cluster'] = agg.fit_predict(rfm_scaled)
print(rfm['Agg_Cluster'].value_counts().sort_index())
# sklearn AgglomerativeClustering gives same result as scipy fcluster
# Compare: rfm[['Hier_Cluster','Agg_Cluster']].value_counts()
```

sklearn AgglomerativeClustering is more convenient for integration with sklearn pipelines; scipy gives more diagnostic information (dendrogram).

#141 Adjusted Rand Index (ARI)

DEFINITION

The Adjusted Rand Index (ARI) measures the similarity between two clustering solutions, adjusted for chance. It ranges from –1 to +1: ARI=1 means perfect agreement; ARI=0 means random agreement; ARI<0 means worse than random. ARI is used to: (1) compare clustering methods, (2) compare a clustering to ground truth labels, and (3) evaluate cluster stability.

FORMULA

ARI = (RI – E[RI]) / (max(RI) – E[RI])
RI = Rand Index (fraction of agreement pairs)

WHEN TO USE

To compare two clustering solutions or evaluate clustering against known labels.

PYTHON CODE

```
from sklearn.metrics import adjusted_rand_score

# Compare K-Means vs Hierarchical
ari = adjusted_rand_score(rfm['Cluster'], rfm['Hier_Cluster'])
print(f'ARI between K-Means and Hierarchical: {ari:.4f}')
# ARI interpretation
if ari > 0.7:
    print('Strong agreement between methods')
elif ari > 0.4:
    print('Moderate agreement')
else:
    print('Methods produce quite different clusterings')
```

ARI > 0.8 between K-Means and Hierarchical clustering indicates both methods find the same natural structure in the data.

#142 Davies-Bouldin Index

DEFINITION

The Davies-Bouldin Index (DBI) measures clustering quality by computing the ratio of within-cluster scatter to between-cluster separation. Lower DBI = better clustering. Unlike Silhouette Score, it is faster to compute for large datasets. DBI = 0 indicates perfect clustering.

FORMULA

DBI = (1/k) Σ■ max_{j≠i} [(σ■+σ■)/d(μ■,μ■)]
σ■ = avg distance within cluster i, d(μ■,μ■) = centroid distance

WHEN TO USE

As an additional clustering quality metric alongside Silhouette Score.

PYTHON CODE

```
from sklearn.metrics import davies_bouldin_score

dbi_scores = []
K_range = range(2, 11)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(rfm_scaled)
    dbi_scores.append(davies_bouldin_score(rfm_scaled, labels))

best_k_dbi = K_range[np.argmin(dbi_scores)]
print(f'Best k by DBI: {best_k_dbi} (DBI={min(dbi_scores):.4f})')
```

Compare optimal k from Elbow, Silhouette, and DBI. If all three agree, confidence in the chosen k is high.

#143 DBSCAN Algorithm

DEFINITION

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) groups points that are close together in dense regions and marks points in sparse regions as noise (outliers). It finds clusters of arbitrary shape without requiring k to be specified. Key parameters: eps (neighborhood radius) and min_samples (minimum points to form a dense region).

FORMULA

Core point: has ≥ min_samples points within eps radius
Border point: within eps of a core point
Noise: neither core nor border (labeled as -1)

WHEN TO USE

For irregularly shaped clusters or when you expect noise points in the data.

PYTHON CODE

```
from sklearn.cluster import DBSCAN

db = DBSCAN(eps=0.5, min_samples=5)
rfm['DBSCAN_Cluster'] = db.fit_predict(rfm_scaled)
n_clusters = len(set(rfm['DBSCAN_Cluster'])) - (1 if -1 in rfm['DBSCAN_Cluster'].values else 0)
n_noise = (rfm['DBSCAN_Cluster'] == -1).sum()
print(f'DBSCAN: {n_clusters} clusters, {n_noise} noise points ({n_noise/len(rfm)*100:.1f}%)')
```

Noise points (-1) are potential outliers in the customer base. DBSCAN cluster count can vary widely with eps and min_samples.

#144 DBSCAN Parameters (eps, min_samples)

DEFINITION

eps is the radius of the neighborhood around each point — larger eps = more points included per neighborhood = fewer, larger clusters. min_samples is the minimum number of points required in an eps-neighborhood to classify a point as a core point — higher min_samples = stricter density requirement = more noise points. Choosing these parameters is the main challenge of DBSCAN.

FORMULA

eps: domain-specific; often chosen from k-distance graph

min_samples ≥ ln(n) is a common starting heuristic

WHEN TO USE

As tuning step before running DBSCAN.

PYTHON CODE

```
# Grid search over eps and min_samples
from sklearn.metrics import silhouette_score
results = []
for eps in [0.3, 0.5, 0.7, 1.0, 1.5]:
    for ms in [3, 5, 10]:
        db = DBSCAN(eps=eps, min_samples=ms)
        labels = db.fit_predict(rfm_scaled)
        n_cl = len(set(labels)) - (1 if -1 in labels else 0)
        if n_cl >= 2 and labels[labels >= 0].size > 0:
            sil = silhouette_score(rfm_scaled[labels >= 0], labels[labels >= 0])
            results.append({'eps':eps,'ms':ms,'n_clusters':n_cl,'silhouette':sil})
print(pd.DataFrame(results).sort_values('silhouette', ascending=False).head())
```

Start with eps from the k-distance elbow graph. Adjust min_samples based on how much noise is acceptable.

#145 k-Distance Graph

DEFINITION

The k-distance graph plots, for each point, its distance to its k-th nearest neighbor (sorted in descending order). The 'elbow' of this curve is the recommended eps value for DBSCAN. Points before the elbow (high distances) become noise; points after the elbow (low distances) become core points. k is typically set to min_samples – 1.

FORMULA

d_k(x■) = distance from x■ to its k-th nearest neighbor

WHEN TO USE

Before running DBSCAN, to find a good eps parameter value.

PYTHON CODE

```
from sklearn.neighbors import NearestNeighbors

k = 5  # min_samples - 1
nbrs = NearestNeighbors(n_neighbors=k).fit(rfm_scaled)
distances, _ = nbrs.kneighbors(rfm_scaled)
k_distances = np.sort(distances[:, -1])[::-1]  # k-th neighbor distance, sorted desc

plt.figure(figsize=(10, 5))
plt.plot(k_distances, color='steelblue')
plt.xlabel('Points (sorted by decreasing distance)')
plt.ylabel(f'{k}-th Nearest Neighbor Distance')
plt.title('k-Distance Graph for DBSCAN eps Selection')
plt.grid(alpha=0.3); plt.show()
```

■ ***The x-value at the elbow of this graph is a good starting point for eps. Try eps values in a range around this elbow value.***

#146 Core Points, Border Points, and Noise

DEFINITION

DBSCAN classifies each point into three types. Core points have at least min_samples points within eps radius — they form the interior of clusters. Border points are within eps of a core point but have fewer than min_samples neighbors — they belong to a cluster but aren't core themselves. Noise points are neither core nor border — they are outliers, labeled -1 by sklearn.

FORMULA

Core: $|N_{eps}(p)| \geq min\_samples$
Border: $|N_{eps}(p)| < min\_samples$, $\exists$ core point c with $d(p,c) \leq eps$
Noise: all others

WHEN TO USE

For understanding the composition of DBSCAN results and identifying outliers.

PYTHON CODE

```
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.5, min_samples=5)
labels = db.fit_predict(rfm_scaled)
core_mask = np.zeros(len(rfm_scaled), dtype=bool)
core_mask[db.core_sample_indices_] = True
noise_mask = labels == -1
border_mask = ~core_mask & ~noise_mask
print(f'Core: {core_mask.sum()}, Border: {border_mask.sum()}, Noise: {noise_mask.sum()}')
```

High noise count (>20%) suggests eps is too small or min_samples too large. Noise points deserve separate business analysis — they may be valuable outliers.

#147 Comparing Clustering Algorithms

DEFINITION

Different clustering algorithms make different assumptions and excel in different scenarios. K-Means: fast, assumes spherical equal-size clusters, sensitive to outliers. Hierarchical: no k needed, slow for large datasets, provides dendrogram. DBSCAN: finds arbitrary shapes, handles noise, sensitive to parameter choice. Comparing them with ARI and cluster profiles determines which method is most appropriate for the data.

FORMULA

N/A

WHEN TO USE

When there is no prior knowledge about cluster structure — run multiple methods and compare.

PYTHON CODE

```
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
methods = {
    'KMeans(k=4)': KMeans(n_clusters=4, random_state=42).fit_predict(rfm_scaled),
    'Hierarchical(k=4)': AgglomerativeClustering(n_clusters=4).fit_predict(rfm_scaled),
}
for m1, l1 in methods.items():
    for m2, l2 in methods.items():
        if m1 < m2:
            ari = adjusted_rand_score(l1, l2)
            print(f'ARI({m1}, {m2}) = {ari:.4f}')
```

High ARI between methods confirms a robust cluster structure. Low ARI means the clusters depend on algorithmic assumptions, not data structure.

#148 Cluster Visualization (2D/3D Scatter)

DEFINITION

Visualizing clusters in 2D (using PCA or t-SNE reduction) or 3D provides intuitive confirmation of cluster quality. Overlapping clusters in PCA space suggest poor separation. t-SNE (t-distributed Stochastic Neighbor Embedding) is better at preserving local structure for visualization but should not be used for distances — only for visualization.

FORMULA

N/A

WHEN TO USE

After any clustering, to visually inspect cluster quality and separation.

PYTHON CODE

```
pca = PCA(n_components=2, random_state=42)
pca_coords = pca.fit_transform(rfm_scaled)

fig, ax = plt.subplots(figsize=(8, 6))
scatter = ax.scatter(pca_coords[:,0], pca_coords[:,1],
                     c=rfm['Cluster'], cmap='Set2',
                     alpha=0.7, s=40)
plt.colorbar(scatter, label='Cluster')
ax.set_title(f'K-Means Clusters in PCA Space
(Explained Var: {pca.explained_variance_ratio_.sum():.2%})')
ax.set_xlabel('PC1'); ax.set_ylabel('PC2')
plt.show()
```

Well-separated color regions = distinct clusters. Bleeding colors = poor cluster separation — consider different k or feature engineering.

#149 Business Interpretation of Clusters

DEFINITION

The final and most critical step: translating cluster profiles into actionable business strategies. Each cluster should receive: (1) a descriptive name reflecting its dominant characteristics, (2) a strategic recommendation, and (3) key metrics that define it. A cluster interpretation table is the primary output presented to stakeholders.

FORMULA

N/A

WHEN TO USE

After cluster profiling — this is the deliverable that justifies the clustering analysis.

PYTHON CODE

```
# Build interpretation table
profile = rfm.groupby('Cluster')[['Recency','Frequency','Monetary']].mean().round(1)
profile['Count'] = rfm['Cluster'].value_counts().sort_index()
profile['Pct'] = (profile['Count'] / len(rfm) * 100).round(1)
profile['Strategy'] = ['Win-back campaign', 'VIP loyalty program',
                       'Upsell opportunities', 'Engagement campaigns']
print(profile.to_string())
```

Every cluster analysis must answer: 'What should we DO differently for each segment?' Without action, clustering is just an academic exercise.

Chapter 5

Time Series Forecasting

ARIMA, SARIMA, Prophet, metrics

46 Concept

#150 Time Series Introduction

DEFINITION

A time series is a sequence of observations recorded at successive equally-spaced time intervals. Examples: daily sales revenue, monthly temperature, hourly website traffic. Unlike cross-sectional data, observations in a time series are NOT independent — each value is related to its past values (autocorrelation). This temporal dependence is the central feature that all time-series methods exploit.

FORMULA

X■ = f(t, X■■■, X■■■, ..., ε■)

WHEN TO USE

Any time the goal involves prediction of future values, or detecting temporal patterns.

PYTHON CODE

```
df_ts = df.copy()
df_ts['order_date'] = pd.to_datetime(df_ts['order_date'])
df_ts = df_ts.set_index('order_date')
ts = df_ts['revenue'].resample('M').sum()
ts.plot(figsize=(14, 5), title='Monthly Revenue Time Series')
plt.ylabel('Revenue'); plt.grid(alpha=0.3); plt.show()
```

■ ***Always visualize the raw time series first. Look for: upward/downward trend, regular seasonal peaks, sudden structural breaks, and obvious outliers.***

#151 Time Series Components

DEFINITION

A time series is typically decomposed into four components: Trend (T) — long-term increase or decrease in the level. Seasonality (S) — regular periodic fluctuations at a fixed frequency (e.g., holiday spikes every December). Cyclicality (C) — irregular multi-year oscillations not fixed in period (e.g., business cycles). Irregular/Noise (I) — random variation unexplained by other components.

FORMULA

Additive: X■ = T + S + C + I■
Multiplicative: X■ = T × S × C × I■

WHEN TO USE

As the conceptual framework before any time-series modeling.

PYTHON CODE

```
from statsmodels.tsa.seasonal import seasonal_decompose

decomp = seasonal_decompose(ts.fillna(ts.interpolate()),
                            model='additive', period=12)
fig = decomp.plot()
fig.set_size_inches(14, 10)
fig.suptitle('Additive Decomposition: Monthly Revenue', y=1.01)
plt.tight_layout(); plt.show()
```

■ ***If the trend and seasonal components are visually larger than the residual, the decomposition captured most of the signal.***

#152 Additive vs Multiplicative Models

DEFINITION

In an additive model, the seasonal variation is constant in magnitude regardless of the trend level — appropriate when seasonal swings are roughly the same size across all periods. In a multiplicative model, seasonal variation grows with the trend level (percentage-based swings) — appropriate when peaks become larger as the series grows. Revenue often follows a multiplicative pattern.

FORMULA

Additive: X■ = T + S + ε
Multiplicative: X■ = T × S × ε

WHEN TO USE

Use multiplicative when seasonal amplitude grows with the trend level (as seen in the decomposition plot).

PYTHON CODE

```
# Additive decomposition
decomp_add = seasonal_decompose(ts.interpolate(), model='additive', period=12)
# Multiplicative decomposition
decomp_mul = seasonal_decompose(ts.interpolate(), model='multiplicative', period=12)
# Compare residual variance
print(f'Additive residual std: {decomp_add.resid.std():.2f}')
print(f'Multiplicative residual std: {decomp_mul.resid.std():.2f}')
# Smaller residual std → better model fit
```

■ ***Choose the model with smaller residual variance. Alternatively, log-transforming the series converts multiplicative patterns to additive.***

#153 Time Series Indexing with DatetimeIndex

DEFINITION

pandas DatetimeIndex enables powerful time-based selection, slicing, and resampling. Setting the DataFrame index to a datetime column with df.set_index() + resample() allows operations like: monthly aggregation, year-over-year comparison, and time-range slicing. PeriodIndex is an alternative for fixed-frequency data.

FORMULA

N/A

WHEN TO USE

Whenever working with time-ordered data in pandas.

PYTHON CODE

```
ts_indexed = df.set_index('order_date').sort_index()
# Time-range slicing
ts_2022 = ts_indexed['2022']
ts_q3 = ts_indexed['2022-07':'2022-09']
# Resampling
monthly = ts_indexed['revenue'].resample('M').sum()
quarterly = ts_indexed['revenue'].resample('Q').mean()
print(monthly.head())
```

DatetimeIndex slicing is inclusive on both ends when using string slice notation ('2022' selects the entire year 2022).

#154 Resampling and Frequency Conversion

DEFINITION

Resampling changes the time-series frequency. Downsampling (aggregating from higher to lower frequency, e.g., daily → monthly) uses aggregation functions: sum, mean, max, etc. Upsampling (from lower to higher frequency, e.g., monthly → daily) creates new timestamps that need to be filled with interpolation or forward/backward fill.

FORMULA

N/A

WHEN TO USE

To align datasets at different frequencies, or to smooth high-frequency noise.

PYTHON CODE

```
daily_ts = df.set_index('order_date')['revenue'].resample('D').sum()
# Monthly sum (downsampling)
monthly_ts = daily_ts.resample('M').sum()
# Upsample to daily and interpolate
upsampled = monthly_ts.resample('D').interpolate(method='linear')
print(f'Monthly: {len(monthly_ts)} periods')
print(f'Upsampled daily: {len(upsampled)} periods')
```

When upsampling, always prefer business-logical fill methods (forward fill for prices, interpolation for volumes).

#155 Handling Missing Timestamps

DEFINITION

Time series often have gaps — missing dates due to weekends (for daily data), sensor downtime, or data pipeline failures. These gaps must be addressed before modeling. Strategies: reindex to a complete date range and fill NaN with 0 (for event counts), forward fill (for prices), linear interpolation (for smooth signals), or 0-fill (for sales on non-business days).

FORMULA

N/A

WHEN TO USE

Before any time-series modeling, to ensure a complete regular time index.

PYTHON CODE

```
# Create complete monthly index
full_index = pd.date_range(ts.index.min(), ts.index.max(), freq='M')
ts_complete = ts.reindex(full_index)
print(f'Missing timestamps: {ts_complete.isna().sum()}')
ts_filled = ts_complete.fillna(method='ffill').fillna(method='bfill')
# Or interpolate:
ts_interp = ts_complete.interpolate(method='time')
```

For daily sales data, 0-fill on weekends/holidays is usually correct. For monthly data, interpolation is more appropriate.

#156 STL Decomposition

DEFINITION

STL (Seasonal and Trend decomposition using Loess) is a robust decomposition method that uses locally weighted regression (Loess/Lowess) to extract trend and seasonal components. Unlike classical decomposition, STL handles any seasonal period, is robust to outliers, and allows the seasonal component to change over time. It is the most flexible and recommended decomposition method.

FORMULA

X■ = T■ + S■ + R■ (additive)
Trend via Loess; Seasonal via inner-loop; Residual = X – T – S

WHEN TO USE

For robust decomposition when data has outliers or changing seasonality, and as a preprocessing step before ARIMA/Prophet.

PYTHON CODE

```
from statsmodels.tsa.seasonal import STL

stl = STL(ts.interpolate(), period=12, robust=True)
result = stl.fit()
fig = result.plot()
fig.set_size_inches(14, 10)
plt.suptitle('STL Decomposition (Robust)', y=1.01)
plt.tight_layout(); plt.show()
print(f'Trend range: {result.trend.min():.0f} – {result.trend.max():.0f}')
```

STL with robust=True down-weights outliers during fitting. The residuals should look like white noise for a good decomposition.

#157 Trend Strength

DEFINITION

Trend strength from STL decomposition quantifies what fraction of the series' total variance is attributable to the trend component. A value close to 1 means the trend is the dominant driver; close to 0 means the series is mostly noise/seasonal without a trend. This metric guides whether trend modeling is necessary.

FORMULA

F_T = max(0, 1 – Var(R■) / Var(T■ + R■))

WHEN TO USE

After STL decomposition, to determine if a trend component warrants modeling.

PYTHON CODE

```
from statsmodels.tsa.seasonal import STL

stl = STL(ts.interpolate(), period=12)
result = stl.fit()
trend_strength = max(0, 1 - result.resid.var() / (result.trend + result.resid).var())
print(f'Trend strength: {trend_strength:.4f}')
if trend_strength > 0.6:
    print('Strong trend present — model must capture trend')
else:
    print('Weak trend — focus on seasonality/cyclicality')
```

■ ***Trend strength > 0.6 = strong trend; > 0.9 = dominant trend. Most retail revenue series have trend strength 0.5–0.85.***

#158 Seasonality Strength

DEFINITION

Seasonality strength measures what fraction of total series variance is explained by the seasonal component (after removing trend). Close to 1 = strongly seasonal (e.g., retail has holiday peaks). Close to 0 = weak or no seasonality. This guides whether seasonal models (SARIMA, Holt-Winters) are needed.

FORMULA

F_S = max(0, 1 – Var(R■) / Var(S■ + R■))

WHEN TO USE

After STL decomposition, to determine if a seasonal model is warranted.

PYTHON CODE

```
seasonality_strength = max(0, 1 - result.resid.var() / (result.seasonal + result.resid).var())
print(f'Seasonality strength: {seasonality_strength:.4f}')
if seasonality_strength > 0.6:
    print('Strong seasonality — use SARIMA or Prophet with yearly_seasonality=True')
else:
    print('Weak seasonality — simple ARIMA may suffice')
```

■ ***Seasonality > 0.6 → SARIMA or Holt-Winters seasonal parameters are worth including.***

#159 Classical Decomposition (seasonal_decompose)

DEFINITION

Classical decomposition splits a time series into trend, seasonal, and residual components using moving average smoothing for trend extraction and averaging aligned seasonal indices for the seasonal component. It is simpler and faster than STL but assumes constant seasonality and handles outliers poorly. Suitable for clean, regular time series.

FORMULA

Moving average trend, then seasonal = X/T or X–T per period

WHEN TO USE

For a quick first look at decomposition; use STL for more robust results.

PYTHON CODE

```
from statsmodels.tsa.seasonal import seasonal_decompose

decomp = seasonal_decompose(ts.fillna(ts.median()),
                            model='multiplicative',
                            period=12)
decomp.plot()
plt.suptitle('Classical Multiplicative Decomposition', y=1.01)
plt.tight_layout(); plt.show()
print('Seasonal indices:')
print(decomp.seasonal.iloc[:12].round(3).values)
```

Seasonal index > 1 means that month is above average (e.g., 1.25 = 25% above trend level). Index < 1 means below average.

#160 ADF Test (Augmented Dickey-Fuller)

DEFINITION

The Augmented Dickey-Fuller (ADF) test checks for unit roots in a time series. A unit root means the series is non-stationary — its statistical properties (mean, variance) change over time. ARIMA requires stationarity. H■: series has a unit root (non-stationary). Reject H■ (p < 0.05) → series is stationary. The 'Augmented' version adds lagged differences to handle autocorrelation.

FORMULA

ΔX■ = α + βt + γX■■■ + Σδ■ ΔX■■■ + ε■

H■: γ = 0 (unit root, non-stationary)

WHEN TO USE

Before fitting ARIMA models, to verify stationarity.

PYTHON CODE

```
from statsmodels.tsa.stattools import adfuller

def adf_test(series, name='Series'):
    result = adfuller(series.dropna(), autolag='AIC')
    print(f'\nADF Test: {name}')
    print(f'  ADF Stat: {result[0]:.4f}')
    print(f'  p-value: {result[1]:.4f}')
    print(f'  {"STATIONARY" if result[1] < 0.05 else "NON-STATIONARY"} (α=0.05)')

adf_test(ts, 'Monthly Revenue')
```

p < 0.05 → reject non-stationarity → series is stationary → ARIMA can be applied directly. p > 0.05 → apply differencing first.

#161 KPSS Test

DEFINITION

The KPSS (Kwiatkowski-Phillips-Schmidt-Shin) test has the opposite null hypothesis from ADF: H■ = series IS stationary. A significant result (p < 0.05) rejects stationarity. Using ADF and KPSS together gives stronger conclusions: ADF rejects non-stationarity AND KPSS does not reject stationarity → strong evidence for stationarity.

FORMULA

H■ (KPSS): series is stationary (around mean or trend)
Test stat based on partial sum of residuals

WHEN TO USE

As a complement to ADF to confirm stationarity from both directions.

PYTHON CODE

```
from statsmodels.tsa.stattools import kpss

def kpss_test(series, name='Series'):
    stat, p, lags, crit = kpss(series.dropna(), regression='c', nlags='auto')
    print(f'\nKPSS Test: {name}')
    print(f'  KPSS Stat: {stat:.4f}, p: {p:.4f}')
    print(f'  {"STATIONARY" if p > 0.05 else "NON-STATIONARY"}')

kpss_test(ts, 'Monthly Revenue')
```

ADF: reject H■ (p<0.05) + KPSS: fail to reject H■ (p>0.05) = strong stationarity evidence. Both significant = possibly trend-stationary.

#162 First-Order Differencing

DEFINITION

Differencing removes trends from a non-stationary time series by computing the difference between consecutive observations: ΔX■ = X■ – X■■■. First differencing often achieves stationarity for series with linear trends. The order of differencing (d) in ARIMA corresponds to how many times differencing is applied.

FORMULA

ΔX■ = X■ – X■■■ (1st order)
Δ²X■ = ΔX■ – ΔX■■■ (2nd order)

WHEN TO USE

When ADF test indicates non-stationarity. Apply d=1 first; rarely need d=2.

PYTHON CODE

```
ts_diff1 = ts.diff().dropna()
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
ts.plot(ax=axes[0], title='Original Series')
ts_diff1.plot(ax=axes[1], title='1st Difference')
plt.tight_layout(); plt.show()
# Re-test ADF on differenced series
adf_test(ts_diff1, '1st Differenced Revenue')
```

After differencing, the series should fluctuate around zero with no visible trend. Re-run ADF to confirm stationarity.

#163 Seasonal Differencing

DEFINITION

Seasonal differencing removes seasonal patterns by subtracting the observation from the same season in the previous period: ∆■X■ = X■ – X■■■ (where s = seasonal period). For monthly data with annual seasonality, s=12. Seasonal differencing handles the 'S' (seasonal) part of non-stationarity that regular first differencing may leave behind.

FORMULA

∆■X■ = X■ – X■■■

For monthly: ∆X■ = X■ – X■■■■

WHEN TO USE

When a series shows strong seasonality that remains after first-order differencing.

PYTHON CODE

```
ts_seasonal_diff = ts.diff(12).dropna()
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
ts.plot(ax=axes[0], title='Original')
ts_seasonal_diff.plot(ax=axes[1], title='Seasonal Differencing (lag=12)')
plt.tight_layout(); plt.show()
adf_test(ts_seasonal_diff, 'Seasonal Differenced')
```

■ ***Seasonal differencing removes the seasonal pattern. If trend remains, apply regular differencing on top of seasonal differencing.***

#164 Log Transformation for Stationarity

DEFINITION

Log transformation (ln(X■)) stabilizes variance in multiplicative time series — it converts exponentially growing trends and multiplicatively varying seasonality into additive form. It also makes the series more symmetric. Log + differencing is a common two-step preprocessing for financial time series.

FORMULA

X_log = ln(X■) [requires X■ > 0]

ln(X■/X■■■) = ln(X■) – ln(X■■■) = log return

WHEN TO USE

When the series has exponential growth or when seasonal amplitude grows with the trend level.

PYTHON CODE

```
ts_log = np.log(ts + 1)  # +1 to handle zeros
ts_log_diff = ts_log.diff().dropna()
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
ts.plot(ax=axes[0], title='Original')
ts_log.plot(ax=axes[1], title='Log Transformed')
ts_log_diff.plot(ax=axes[2], title='Log + Differenced')
plt.tight_layout(); plt.show()
adf_test(ts_log_diff, 'Log-Differenced')
```

■ ***Log-differenced series represents log returns (% changes). This is the standard form for financial series analysis.***

#165 ACF — Autocorrelation Function

DEFINITION

The Autocorrelation Function (ACF) measures the correlation between a time series and lagged versions of itself. ACF(k) = correlation between X■ and X■■■. In the ACF plot, the x-axis is the lag; y-axis is the autocorrelation coefficient. Blue shaded bands = 95% confidence interval. Spikes outside the band indicate significant autocorrelation at that lag.

FORMULA

ACF(k) = Corr(X■, X■■■)
Confidence band ≈ ±1.96/√n

WHEN TO USE

To determine the MA order (q) for ARIMA and to check residual independence after modeling.

PYTHON CODE

```
from statsmodels.graphics.tsaplots import plot_acf

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
plot_acf(ts.dropna(), lags=24, ax=axes[0], title='ACF — Original Series')
ts_diff = ts.diff().dropna()
plot_acf(ts_diff, lags=24, ax=axes[1], title='ACF — Differenced Series')
plt.tight_layout(); plt.show()
```

ACF cutting off after lag q (spikes only at lags 1..q, then zero) → MA(q) process. Slow geometric decay → AR process, needs differencing.

#166 PACF — Partial Autocorrelation Function

DEFINITION

The Partial Autocorrelation Function (PACF) measures the correlation between X■ and X■■■ after removing the effects of intermediate lags 1 through k–1. PACF cuts off abruptly after lag p for an AR(p) process, while ACF decays geometrically. Together, ACF and PACF guide the selection of AR and MA orders in ARIMA.

FORMULA

PACF(k) = Corr(X■, X■■■ | X■■■, ..., X■■■■■■■)

WHEN TO USE

To determine the AR order (p) for ARIMA.

PYTHON CODE

```
from statsmodels.graphics.tsaplots import plot_pacf

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
ts_diff = ts.diff().dropna()
plot_acf(ts_diff, lags=24, ax=axes[0], title='ACF (Differenced)')
plot_pacf(ts_diff, lags=24, ax=axes[1], title='PACF (Differenced)',
          method='ywm')
plt.tight_layout(); plt.show()
```

ACF rule: spike at lag q only → MA(q). PACF rule: spike at lag p only → AR(p). Both decay geometrically → ARMA(p,q).

#167 Lag Selection from ACF/PACF

DEFINITION

The Box-Jenkins methodology uses ACF and PACF to identify ARIMA(p,d,q) orders: (1) d = number of differences to achieve stationarity (from ADF test). (2) p = AR order = lag where PACF cuts off. (3) q = MA order = lag where ACF cuts off. These are initial estimates; information criteria (AIC/BIC) are used to fine-tune.

FORMULA

AR(p): PACF cuts off after lag p, ACF decays
MA(q): ACF cuts off after lag q, PACF decays
ARMA(p,q): both ACF and PACF decay gradually

WHEN TO USE

After achieving stationarity (differencing), to propose candidate ARIMA models.

PYTHON CODE

```
from statsmodels.tsa.stattools import acf, pacf

ts_diff = ts.diff().dropna()
acf_vals = acf(ts_diff, nlags=20)
pacf_vals = pacf(ts_diff, nlags=20, method='ywm')
sig_acf = np.where(np.abs(acf_vals[1:]) > 1.96/np.sqrt(len(ts_diff)))[0] + 1
sig_pacf = np.where(np.abs(pacf_vals[1:]) > 1.96/np.sqrt(len(ts_diff)))[0] + 1
print(f'Significant ACF lags: {sig_acf[:5]}')
print(f'Significant PACF lags: {sig_pacf[:5]}')
```

If PACF has 1 significant spike (at lag 1), start with AR(1). If ACF has 1 spike (at lag 1), try MA(1). Combine as needed.

#168 AR Model (AutoRegressive)

DEFINITION

An AR(p) model regresses the current value on its own p past values. The key assumption is stationarity. AR models are appropriate when the series shows persistent autocorrelation — each period's value depends partly on recent history. AR(1): X■ = c + ϕ■X■■■ + ε■ is the simplest case; ϕ■ close to 1 indicates strong momentum.

FORMULA

X■ = c + ϕ■X■■■ + ϕ■X■■■ + ... + ϕ■X■■■ + ε■
|ϕ■| < 1 for stationarity

WHEN TO USE

When PACF shows sharp cutoff after lag p and ACF decays geometrically.

PYTHON CODE

```
from statsmodels.tsa.ar_model import AutoReg

ts_diff = ts.diff().dropna()
ar_model = AutoReg(ts_diff, lags=2).fit()
print(ar_model.summary())
print(f'AIC: {ar_model.aic:.2f}')
```

AR coefficient close to 1 = high momentum/persistence. Check that all AR roots are outside the unit circle for stationarity.

#169 MA Model (Moving Average)

DEFINITION

An MA(q) model expresses the current value as a linear combination of the current and past q white-noise error terms. Unlike AR models, MA models have finite memory — their effect dies out after q lags. MA(1): X■ = μ + ε■ + θ■ε■■■. MA models are suited for series where shocks (sudden events) have short-lived but delayed effects.

FORMULA

X■ = μ + ε■ + θ■ε■■■ + ... + θqε■■q

ε■ ~ WN(0, σ²)

WHEN TO USE

When ACF shows sharp cutoff after lag q and PACF decays geometrically.

PYTHON CODE

```
from statsmodels.tsa.arima.model import ARIMA

ma_model = ARIMA(ts_diff, order=(0, 0, 1)).fit()
print(ma_model.summary())
print(f'MA(1) AIC: {ma_model.aic:.2f}')
```

MA models are often used for seasonally differenced series. If residuals are white noise (ACF of residuals shows no spikes), MA(q) is adequate.

#170 ARMA Model

DEFINITION

An ARMA(p,q) model combines AR(p) and MA(q) components. It captures both the autoregressive memory (dependence on past values) and the moving average response to past shocks. ARMA requires a stationary series — it is the model for the stationary series after differencing. ARIMA adds the 'I' (Integrated) component for the differencing step.

FORMULA

X■ = c + Σϕ■ X■■■ + ε■ + Σθ■ ε■■■

WHEN TO USE

When both ACF and PACF decay gradually (mixed AR+MA behavior).

PYTHON CODE

```
arma = ARIMA(ts_diff, order=(1, 0, 1)).fit()
print(f'ARMA(1,1): AIC={arma.aic:.2f}, BIC={arma.bic:.2f}')
# Residual diagnostics
arma.plot_diagnostics(figsize=(12, 8))
plt.suptitle('ARMA(1,1) Residual Diagnostics', y=1.01)
plt.tight_layout(); plt.show()
```

Good ARMA residuals: (1) no pattern in residuals vs time, (2) Q-Q plot near diagonal, (3) ACF of residuals shows no significant spikes.

#171 ARIMA Model

DEFINITION

ARIMA(p,d,q) combines differencing (I: Integrated) with AR and MA components. d is the order of differencing applied to achieve stationarity; the model is fit on the differenced series. ARIMA is the workhorse model for non-seasonal time series. Forecasts are automatically de-differenced back to the original scale.

FORMULA

(1 – ϕ■L – ... – ϕ■L■)(1–L)■ X■ = (1 + θ■L + ... + θqLq)ε■

L = lag operator: LX■ = X■■■

WHEN TO USE

For non-seasonal univariate time series forecasting after confirming stationarity requirements.

PYTHON CODE

```
from statsmodels.tsa.arima.model import ARIMA

train, test = ts[:-12], ts[-12:]
model = ARIMA(train, order=(1, 1, 1)).fit()
forecast = model.forecast(steps=12)

plt.figure(figsize=(14, 5))
train.plot(label='Train')
test.plot(label='Test (Actual)', color='green')
forecast.plot(label='ARIMA Forecast', color='red', ls='--')
plt.title('ARIMA(1,1,1) Forecast vs Actual')
plt.legend(); plt.show()
print(model.summary())
```

■ ***Compare forecast vs actual in the test period. Calculate MAE/RMSE to quantify accuracy.***

#172 AIC and BIC for Model Selection

DEFINITION

AIC (Akaike Information Criterion) and BIC (Bayesian Information Criterion) balance model fit (log-likelihood) against model complexity (number of parameters). Lower AIC/BIC = better model. BIC penalizes complexity more strongly than AIC, tending to favor simpler models. Use them to compare ARIMA(p,d,q) candidates with different orders.

FORMULA

AIC = 2k – 2ln(L■)
BIC = k·ln(n) – 2ln(L■)
k = parameters, L■ = max log-likelihood

WHEN TO USE

To compare multiple ARIMA models with different (p,q) orders on the same dataset.

PYTHON CODE

```
# Compare models by AIC
results = []
for p in range(3):
    for q in range(3):
        try:
            m = ARIMA(train, order=(p,1,q)).fit()
            results.append({'p':p,'q':q,'AIC':m.aic,'BIC':m.bic})
        except:
            pass
aic_df = pd.DataFrame(results).sort_values('AIC')
print(aic_df.head(5))
```

The model with lowest AIC is preferred for predictive accuracy. Lowest BIC for parsimony (fewer parameters). If they disagree, prefer BIC for smaller samples.

#173 auto_arima (pmdarima)

DEFINITION

auto_arima() from the pmdarima library automatically searches over combinations of p, d, q (and P, D, Q, s for seasonal models) and selects the best-fitting ARIMA/SARIMA model by AIC. It performs ADF tests internally to determine d, and uses a stepwise search algorithm to make the process computationally feasible.

FORMULA

N/A (automated grid search over ARIMA orders by AIC)

WHEN TO USE

When you want to automate ARIMA order selection without manual ACF/PACF inspection.

PYTHON CODE

```
from pmdarima import auto_arima

auto_model = auto_arima(train,
                        seasonal=False,
                        stepwise=True,
                        information_criterion='aic',
                        trace=True,
                        error_action='ignore',
                        suppress_warnings=True)
print(auto_model.summary())
forecast_auto = auto_model.predict(n_periods=12)
```

auto_arima is a good starting point but always validate its choice with residual diagnostics. It may not find the globally optimal model.

#174 SARIMA Model

DEFINITION

SARIMA (Seasonal ARIMA) extends ARIMA with seasonal AR(P), differencing(D), and MA(Q) components at the seasonal lag s: SARIMA(p,d,q)(P,D,Q)s. For monthly data with annual seasonality, s=12. The seasonal components capture patterns that repeat every s periods. D=1 means seasonal differencing is applied; P and Q are seasonal AR and MA orders.

FORMULA

SARIMA(p,d,q)(P,D,Q)s:
(1−Φ■L■)...(1−L■)■ (1−ϕ■L)...(1−L)■ X■ = (1+Θ■L■)...(1+θ■L)...ε■

WHEN TO USE

For seasonal time series where simple ARIMA leaves significant autocorrelation at seasonal lags.

PYTHON CODE

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

sarima = SARIMAX(train,
                 order=(1, 1, 1),
                 seasonal_order=(1, 1, 1, 12)).fit(disp=False)
forecast_sarima = sarima.forecast(steps=12)
print(f'SARIMA AIC: {sarima.aic:.2f}')
print('Forecast:', forecast_sarima.values.round(0))
```

SARIMA's seasonal MA(1,12) parameter (Θ■) captures the seasonal shock pattern. Positive Θ■ = seasonal momentum.

#175 SARIMAX (with Exogenous Variables)

DEFINITION

SARIMAX extends SARIMA with exogenous variables (X) — external predictors that influence the time series. Examples: holiday dummy variables, promotional flags, economic indicators. The exogenous variables must be available for both training and forecast periods. Adding meaningful exogenous variables often significantly reduces forecast error.

FORMULA

SARIMA model + β'Z■ (Z■ = exogenous variables)

WHEN TO USE

When known external factors (promotions, holidays, weather) influence the time series.

PYTHON CODE

```
# Create holiday dummy (November/December = holiday season)
holiday = (ts.index.month.isin([11, 12])).astype(int)
sarimax = SARIMAX(train,
                  exog=pd.Series(holiday[:len(train)]),
                  order=(1,1,1),
                  seasonal_order=(1,1,1,12)).fit(disp=False)
# Forecast with future exogenous values
future_holiday = holiday[-12:].values
forecast_sarimax = sarimax.forecast(steps=12, exog=future_holiday)
```

■ ***If the holiday dummy coefficient is positive and significant, revenue is significantly higher in November/December.***

#176 Train/Test Split for Time Series

DEFINITION

Unlike cross-sectional data, time series must be split chronologically — training data must precede test data, never mixed randomly. A common split: use the last 12 months (or 20% of periods) as the test set and everything before as training. This simulates the real forecasting scenario where future values are unknown at training time.

FORMULA

Train: [t■, ..., t_{n-h}]
Test: [t_{n-h+1}, ..., t_n]
h = forecast horizon

WHEN TO USE

Before fitting any forecasting model, to ensure honest out-of-sample evaluation.

PYTHON CODE

```
h = 12  # forecast horizon (last 12 months)
train, test = ts[:-h], ts[-h:]
print(f'Train: {len(train)} periods ({train.index[0]} – {train.index[-1]})')
print(f'Test:  {len(test)} periods ({test.index[0]} – {test.index[-1]})')
# Verify no overlap
assert train.index[-1] < test.index[0], 'Data leakage!'
```

■ ***Never use future test data to select model parameters. All preprocessing, scaling, and imputation must be fit only on training data.***

#177 Walk-Forward (Expanding Window) Validation

DEFINITION

Walk-forward validation iteratively expands the training window by one period, fits the model, and forecasts one step ahead. This more accurately simulates real-world forecasting than a single train/test split, capturing model performance as new data accumulates. It produces a distribution of errors across time, revealing whether model performance degrades in certain periods.

FORMULA

For each t from n_train to n–1:
Fit on [1..t], forecast t+1, compute error

WHEN TO USE

For rigorous time-series model evaluation, especially for deployment decisions.

PYTHON CODE

```
from sklearn.metrics import mean_absolute_error

errors = []
min_train = 24  # minimum 24 months to start
for end in range(min_train, len(ts) - 1):
    train_wf = ts.iloc[:end]
    actual = ts.iloc[end]
    try:
        m = ARIMA(train_wf, order=(1,1,1)).fit()
        pred = m.forecast(1).values[0]
        errors.append(abs(pred - actual))
    except:
        pass
print(f'Walk-Forward MAE: {np.mean(errors):.2f}')
```

Walk-forward MAE is more honest than in-sample MAE. If it is much larger than in-sample MAE, the model is overfitting to training data.

#178 MAE — Mean Absolute Error

DEFINITION

MAE is the average of absolute differences between forecasted and actual values. It treats all errors equally regardless of direction and is easy to interpret in the units of the target variable. MAE is less sensitive to large errors than RMSE, making it more robust when the test set contains outliers or unusual periods.

FORMULA

MAE = (1/n) Σ|y■ – ■■|

WHEN TO USE

As the primary forecast accuracy metric when outliers in the test period should not have disproportionate influence.

PYTHON CODE

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(test.values, forecast_sarima.values)
print(f'MAE: {mae:.2f}')
# As % of mean:
print(f'MAE as % of mean: {mae/test.mean()*100:.1f}%')
```

MAE of 15 on a series with mean 200 = 7.5% error (MAPE). Lower is better. No universal threshold — benchmark against a naive model.

#179 RMSE — Root Mean Squared Error

DEFINITION

RMSE is the square root of the average squared forecast errors. By squaring errors before averaging, RMSE penalizes large errors much more heavily than MAE. RMSE is in the same units as the target and is sensitive to outliers. It is the most common metric in forecasting competitions and academic papers.

FORMULA

RMSE = √((1/n) Σ(y■ – ■■)²)

WHEN TO USE

When large forecast errors are disproportionately costly (e.g., overstocking/understocking in supply chains).

PYTHON CODE

```
from sklearn.metrics import mean_squared_error

rmse = np.sqrt(mean_squared_error(test.values, forecast_sarima.values))
print(f'RMSE: {rmse:.2f}')
# RMSE > MAE always. If RMSE >> MAE, large errors are occurring
ratio = rmse / mean_absolute_error(test.values, forecast_sarima.values)
print(f'RMSE/MAE ratio: {ratio:.2f} (>1.5 = large errors present)')
```

■ ***RMSE/MAE ratio > 2 means there are a few very large forecast errors dominating. Investigate those periods for structural breaks.***

#180 MAPE — Mean Absolute Percentage Error

DEFINITION

MAPE expresses forecast error as a percentage of the actual value, making it scale-independent and intuitive (e.g., '8% average error'). However, MAPE is undefined when actual values are zero and is asymmetric — under-forecasting is penalized more than over-forecasting. Not recommended for intermittent demand series.

FORMULA

MAPE = (100/n) Σ |y■ – ■■| / y■

WHEN TO USE

When communicating accuracy to business stakeholders who understand percentages better than absolute units.

PYTHON CODE

```
def mape(actual, forecast):
    mask = actual != 0
    return 100 * np.mean(np.abs((actual[mask] - forecast[mask]) / actual[mask]))

mape_val = mape(test.values, forecast_sarima.values)
print(f'MAPE: {mape_val:.2f}%')
# Industry benchmarks:
# < 10%: excellent; 10-20%: good; 20-50%: reasonable; > 50%: poor
```

■ ***MAPE < 10% is considered excellent for most business forecasting. For high-volatility series, 15–25% may be reasonable.***

#181 SMAPE — Symmetric MAPE

DEFINITION

SMAPE addresses the asymmetry of MAPE by using the average of actual and forecast in the denominator instead of just actual. It is symmetric: over-forecasting and under-forecasting of equal magnitude produce the same SMAPE. SMAPE also handles near-zero values better than MAPE (though not zero values). Range: 0% to 200%.

FORMULA

SMAPE = (100/n) Σ 2|y■–■■| / (|y■|+|■■|)

WHEN TO USE

When series contains near-zero values or when symmetric treatment of over/under forecasting is required.

PYTHON CODE

```
def smape(actual, forecast):
    denom = (np.abs(actual) + np.abs(forecast)) / 2
    mask = denom != 0
    return 100 * np.mean(np.abs(actual[mask] - forecast[mask]) / denom[mask])

smape_val = smape(test.values, forecast_sarima.values)
print(f'SMAPE: {smape_val:.2f}%')
```

SMAPE = 0% → perfect forecast. SMAPE = 200% → catastrophic forecast. SMAPE was used as the metric in many M-competition forecasting benchmarks.

#182 Forecast Confidence Intervals

DEFINITION

Forecast confidence intervals express uncertainty in predictions. A 95% PI (Prediction Interval) means that 95% of future observations should fall within the bounds, given the model. Intervals widen with forecast horizon — uncertainty compounds over time. Plotting CIs alongside point forecasts communicates the range of plausible future outcomes.

FORMULA

PI = ■■■■ ± z_(α/2) × σ_h

σ_h increases with forecast horizon h

WHEN TO USE

Always include CIs in forecasts — a point forecast without uncertainty bounds is incomplete.

PYTHON CODE

```
# Get forecast with CI from SARIMA
forecast_obj = sarima.get_forecast(steps=12)
fc_mean = forecast_obj.predicted_mean
fc_ci = forecast_obj.conf_int(alpha=0.05)

plt.figure(figsize=(14, 5))
train[-24:].plot(label='Training Data')
test.plot(label='Actual', color='green')
fc_mean.plot(label='Forecast', color='red')
plt.fill_between(fc_ci.index, fc_ci.iloc[:,0], fc_ci.iloc[:,1],
                 alpha=0.2, color='red', label='95% CI')
plt.legend(); plt.title('SARIMA Forecast with 95% PI')
plt.show()
```

Actual values falling outside the 95% CI more than 5% of the time indicates model misspecification.

#183 Residual Diagnostics for Time Series

DEFINITION

For a good time-series model, residuals should behave like white noise: (1) zero mean, (2) constant variance, (3) no autocorrelation (ACF of residuals shows no significant spikes), (4) approximately normal distribution. The Ljung-Box test formally tests whether residuals are white noise. statsmodels .plot_diagnostics() visualizes all four diagnostics.

FORMULA

White noise: ε■ ~ iid N(0, σ²)
Residuals = y■ – ■■

WHEN TO USE

After fitting any time-series model, before accepting it for forecasting.

PYTHON CODE

```
model = sarima
fig = model.plot_diagnostics(figsize=(12, 8))
plt.suptitle('SARIMA Residual Diagnostics', y=1.01)
plt.tight_layout()
# The 4 panels: Standardized Residuals, Histogram+KDE, Q-Q, ACF
plt.show()
# Check residual ACF
from statsmodels.graphics.tsaplots import plot_acf
plot_acf(model.resid, lags=24)
plt.title('ACF of Residuals — should show no significant spikes')
plt.show()
```

■ ***If residual ACF has spikes beyond the confidence bands, the model is leaving autocorrelation unexplained — increase p or q.***

#184 Ljung-Box Test

DEFINITION

The Ljung-Box test is a portmanteau test for autocorrelation in residuals. It jointly tests whether the first k autocorrelations of the residuals are all zero. H■: the residuals are white noise (no autocorrelation). A significant result (p < 0.05) indicates remaining autocorrelation — the model is inadequate and needs more AR/MA terms.

FORMULA

Q = n(n+2) Σ■■■■ r²■/(n–k) ~ χ²(K) under H■
r■ = autocorrelation of residuals at lag k

WHEN TO USE

As the formal test for residual white noise after fitting ARIMA/SARIMA.

PYTHON CODE

```
from statsmodels.stats.diagnostic import acorr_ljungbox

lb_result = acorr_ljungbox(sarima.resid, lags=12, return_df=True)
print(lb_result)
# Check all p-values
if (lb_result['lb_pvalue'] > 0.05).all():
    print('Residuals are white noise at all lags — model is adequate')
else:
    print('Autocorrelation remains in residuals — model needs improvement')
```

■ ***All Ljung-Box p-values > 0.05 at the first 20 lags confirms white-noise residuals, validating the model.***

#185 Facebook Prophet — Introduction

DEFINITION

Prophet is an open-source forecasting model from Facebook (Meta) designed for business time series with strong seasonality, holiday effects, and missing data. It decomposes the series into trend + seasonality + holidays + error, fitting the components using Stan (Bayesian regression). It is more automated than ARIMA and especially effective for series with holiday spikes and changing growth rates.

FORMULA

$y(t) = g(t) + s(t) + h(t) + \varepsilon$■

g(t)=trend, s(t)=seasonality, h(t)=holidays

WHEN TO USE

For business time series with complex seasonality, holiday effects, or irregular data.

PYTHON CODE

```
from prophet import Prophet

prophet_df = pd.DataFrame({'ds': ts.index, 'y': ts.values})
m = Prophet(yearly_seasonality=True, weekly_seasonality=False,
            daily_seasonality=False)
m.fit(prophet_df)
future = m.make_future_dataframe(periods=12, freq='M')
forecast = m.predict(future)
print(forecast[['ds','yhat','yhat_lower','yhat_upper']].tail(12))
```

■ ***Prophet is robust to missing data and outliers. It provides interpretable components (trend, seasonality) that are easy to explain to stakeholders.***

#186 Prophet Trend Component

DEFINITION

Prophet models two types of trends: logistic growth (for series with a carrying capacity, like user adoption) and linear/piecewise linear growth (for open-ended business metrics). Changepoints are automatically detected by the model — moments where the trend rate changes. The n_changepoints and changepoint_prior_scale parameters control flexibility.

FORMULA

Linear: $g(t) = k + at$

Logistic: $g(t) = L / (1 + \exp(-k(t-m)))$

with changepoints at selected times

WHEN TO USE

When the time series trend is not perfectly linear — common in business data with growth acceleration or saturation.

PYTHON CODE

```
m = Prophet(growth='linear', changepoint_prior_scale=0.05,
            n_changepoints=25)
m.fit(prophet_df)
future = m.make_future_dataframe(periods=12, freq='M')
fc = m.predict(future)
fig = m.plot_components(fc)
plt.suptitle('Prophet Components', y=1.01)
plt.show()
# Inspect changepoints
print('Detected changepoints:')
print(m.changepoints)
```

■ ***Higher changepoint_prior_scale (e.g., 0.5) → more flexible trend, risk of overfitting. Lower (0.01) → smoother trend, risk of underfitting.***

#187 Prophet Seasonality Components

DEFINITION

Prophet decomposes seasonality into Fourier series — harmonic sinusoidal waves at different frequencies. Yearly seasonality uses 10 Fourier terms by default. Weekly seasonality uses 3. Daily uses 4. The seasonality_prior_scale parameter controls regularization strength — higher values allow more flexible seasonal patterns.

FORMULA

s(t) = Σ■ [a■cos(2πnt/P) + b■sin(2πnt/P)]
P = period (365.25 for yearly, 7 for weekly)

WHEN TO USE

For time series with known recurring seasonal patterns.

PYTHON CODE

```
# Custom seasonality example: quarterly (4-period)
m = Prophet(yearly_seasonality=True)
m.add_seasonality(name='quarterly', period=91.25, fourier_order=5)
m.fit(prophet_df)
fc = m.predict(m.make_future_dataframe(periods=12, freq='M'))
fig = m.plot_components(fc)
plt.show()
```

■ ***More Fourier terms (higher fourier_order) = more flexible seasonal curve, but risk overfitting with short series.***

#188 Prophet Holidays and Special Events

DEFINITION

Prophet allows explicit specification of known holiday dates with optional windows (days before/after). For each holiday, Prophet fits an additive adjustment coefficient. This is particularly valuable for retail where specific dates (Black Friday, Christmas) cause predictable spikes that simple seasonality cannot capture because they shift year-to-year.

FORMULA

h(t) = Σ■ κ■ · ■[t ∈ holiday_i window]
κ■ = holiday effect coefficient

WHEN TO USE

When you have known one-off or recurring special events that create predictable demand spikes.

PYTHON CODE

```
holidays = pd.DataFrame({
    'holiday': ['black_friday'] * 3,
    'ds': pd.to_datetime(['2021-11-26', '2022-11-25', '2023-11-24']),
    'lower_window': -2,
    'upper_window': 1
})
m_hol = Prophet(holidays=holidays, yearly_seasonality=True)
m_hol.fit(prophet_df)
fc_hol = m_hol.predict(m_hol.make_future_dataframe(periods=12, freq='M'))
```

■ ***Prophet holiday coefficients show the average lift/drop during that event. Positive = demand spike; negative = demand drop (e.g., post-holiday slump).***

#189 Prophet Changepoints

DEFINITION

Changepoints are moments in the historical data where Prophet's trend algorithm detects a change in growth rate. They are identified by placing potential changepoints at the first 80% of training data and using a sparse Laplace prior (L1 regularization) to select which ones are significant. The plot_components() function shows the trend with detected changepoints.

FORMULA

k + Σ■ a■(t) δ■
a■(t) = 1 if t ≥ s■, else 0
δ■ ~ Laplace(0, λ) [sparse prior]

WHEN TO USE

To understand structural breaks in a time series trend automatically.

PYTHON CODE

```
# Plot changepoints on forecast
fig = m.plot(fc)
from prophet.plot import add_changepoints_to_plot
a = add_changepoints_to_plot(fig.gca(), m, fc)
plt.title('Prophet Forecast with Changepoints')
plt.show()
# Access changepoint magnitudes
deltas = m.params['delta'].mean(axis=0)
print('Changepoint magnitudes:', deltas.round(3))
```

Large positive delta = trend acceleration at that changepoint. Large negative = trend deceleration. These often correspond to business events (product launches, marketing campaigns).

#190 Prophet Forecasting and Visualization

DEFINITION

After fitting, Prophet's predict() method generates forecasts with uncertainty intervals (yhat_lower, yhat_upper) from Monte Carlo sampling of posterior parameters. The built-in plot() shows the forecast and uncertainty band over historical and future periods. plot_components() shows each component separately for interpretability.

FORMULA

yhat = g(t) + s(t) + h(t)
Uncertainty from posterior sampling of trend + noise

WHEN TO USE

After fitting Prophet, to generate and visualize the final forecast.

PYTHON CODE

```
fig1 = m.plot(fc, figsize=(14, 6))
plt.title('Prophet Forecast')
plt.show()

fig2 = m.plot_components(fc)
plt.suptitle('Prophet Components', y=1.01)
plt.tight_layout(); plt.show()

# Extract forecast for test period
test_preds = fc[fc['ds'].isin(test.index)]['yhat']
mae_prophet = mean_absolute_error(test.values, test_preds.values[:len(test)])
print(f'Prophet MAE (test): {mae_prophet:.2f}')
```

Compare Prophet test MAE to SARIMA test MAE to choose the better model for deployment.

#191 Prophet Cross-Validation

DEFINITION

Prophet's built-in cross-validation simulates multiple train/test splits using walk-forward validation. Parameters: initial (training period length), period (step between cutoffs), horizon (forecast horizon). It computes performance metrics (MAE, RMSE, MAPE) at each forecast horizon to reveal how accuracy degrades with forecast distance.

FORMULA

Walk-forward CV over multiple cutoff dates

WHEN TO USE

For rigorous performance evaluation before deploying a Prophet model.

PYTHON CODE

```
from prophet.diagnostics import cross_validation, performance_metrics

df_cv = cross_validation(m, initial='730 days', period='180 days',
                         horizon='365 days', parallel='processes')
df_p = performance_metrics(df_cv)
print(df_p[['horizon','mae','rmse','mape']].head(12))
# Plot MAPE by horizon
df_p.set_index('horizon')['mape'].plot(figsize=(8,4))
plt.title('Prophet MAPE by Forecast Horizon')
plt.ylabel('MAPE'); plt.xlabel('Horizon (days)'); plt.show()
```

MAPE increasing with horizon is expected — uncertainty compounds. A sharp jump at a specific horizon may indicate a structural break.

#192 Exponential Smoothing (Holt-Winters)

DEFINITION

Holt-Winters Exponential Smoothing is a classical forecasting method that uses weighted averages of past observations, giving more weight to recent data (controlled by smoothing parameters α, β, γ). Three versions: Simple ES (level only), Holt's (level + trend), Holt-Winters (level + trend + seasonality). It is computationally faster than ARIMA and often competitive in accuracy.

FORMULA

Level: l■ = αy■ + (1–α)(l■■■ + b■■■)
Trend: b■ = β(l■–l■■■) + (1–β)b■■■
Seasonal: s■ = γ(y■/l■) + (1–γ)s■■m

WHEN TO USE

When interpretability and speed matter, or as a strong baseline to compare against ARIMA/Prophet.

PYTHON CODE

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing

hw_model = ExponentialSmoothing(train,
                                trend='add',
                                seasonal='add',
                                seasonal_periods=12).fit()
forecast_hw = hw_model.forecast(12)
mae_hw = mean_absolute_error(test.values, forecast_hw.values)
print(f'Holt-Winters MAE: {mae_hw:.2f}')
print(f'α={hw_model.params["smoothing_level"]:.3f}, '
      f'β={hw_model.params["smoothing_trend"]:.3f}, '
      f'γ={hw_model.params["smoothing_seasonal"]:.3f}')
```

α close to 1 = model puts nearly all weight on the most recent observation. α close to 0 = model is very stable (slow to respond to changes).

#193 GARCH Models (Introduction)

DEFINITION

GARCH (Generalized AutoRegressive Conditional Heteroscedasticity) models time-varying volatility in financial time series — periods of high volatility cluster together. GARCH(p,q) models the conditional variance as a function of past squared residuals (ARCH terms) and past conditional variances (GARCH terms). Used for risk management, option pricing, and Value-at-Risk (VaR) estimation.

FORMULA

$\sigma_t^2 = \omega + \sum \alpha_i \varepsilon_{t-i}^2 + \sum \beta_j \sigma_{t-j}^2$

$\varepsilon_t \sim N(0, \sigma_t^2)$ ARCH(q)+GARCH(p)

WHEN TO USE

For financial returns or any series exhibiting volatility clustering (calm periods followed by turbulent periods).

PYTHON CODE

```
from arch import arch_model

returns = ts_log.diff().dropna() * 100  # log returns as percentage
garch = arch_model(returns, vol='GARCH', p=1, q=1)
res = garch.fit(disp='off')
print(res.summary())
# Conditional volatility
res.conditional_volatility.plot(figsize=(12,4),
                                title='Conditional Volatility (GARCH)')
plt.show()
```

High conditional volatility periods in the plot identify the most uncertain/volatile timeframes — important for risk management.

#194 Forecast Comparison (ARIMA vs Prophet vs Holt-Winters)

DEFINITION

Best practice is to compare multiple forecasting models on the same test set using multiple metrics before selecting one for deployment. A comparison table including MAE, RMSE, MAPE, and SMAPE alongside model complexity and interpretability provides a holistic basis for model selection. The simplest model with competitive accuracy should be preferred (Occam's Razor).

FORMULA

N/A

WHEN TO USE

Before deploying any forecasting model — always benchmark against multiple alternatives.

PYTHON CODE

```
# Collect all forecasts
results = {
    'SARIMA': forecast_sarima.values,
    'Prophet': test_preds.values[:len(test)],
    'Holt-Winters': forecast_hw.values
}
actuals = test.values
comparison = []
for name, preds in results.items():
    comparison.append({
        'Model': name,
        'MAE': mean_absolute_error(actuals, preds),
        'RMSE': np.sqrt(mean_squared_error(actuals, preds)),
        'MAPE': mape(actuals, preds)
    })
print(pd.DataFrame(comparison).set_index('Model').round(2))
```

If Prophet and SARIMA have similar MAE but SARIMA is simpler to explain to stakeholders, prefer SARIMA for production deployment.

#195 Deployment Considerations for Time Series Models

DEFINITION

Deploying a forecasting model in production requires: (1) Retraining schedule — how often to retrain as new data arrives. (2) Model monitoring — tracking forecast errors and data drift over time. (3) Fallback strategy — a simpler model to use when the primary model fails. (4) Forecast storage — saving forecasts with timestamps for retrospective analysis. (5) Uncertainty communication — always report prediction intervals, not just point forecasts.

FORMULA

N/A

WHEN TO USE

When transitioning from research/notebook to production pipeline.

PYTHON CODE

```
import json
from datetime import datetime

# Save model metadata
metadata = {
    'model': 'SARIMA(1,1,1)(1,1,1,12)',
    'train_end': str(train.index[-1]),
    'test_mae': float(mean_absolute_error(test.values, forecast_sarima.values)),
    'timestamp': datetime.now().isoformat(),
    'forecast': {str(k): float(v) for k, v in zip(test.index, forecast_sarima)}
}
with open('forecast_metadata.json', 'w') as f:
    json.dump(metadata, f, indent=2)
print('Forecast metadata saved')
```

Always version and timestamp forecasts. Retrospective comparison of stored forecasts vs actuals enables continuous model improvement.